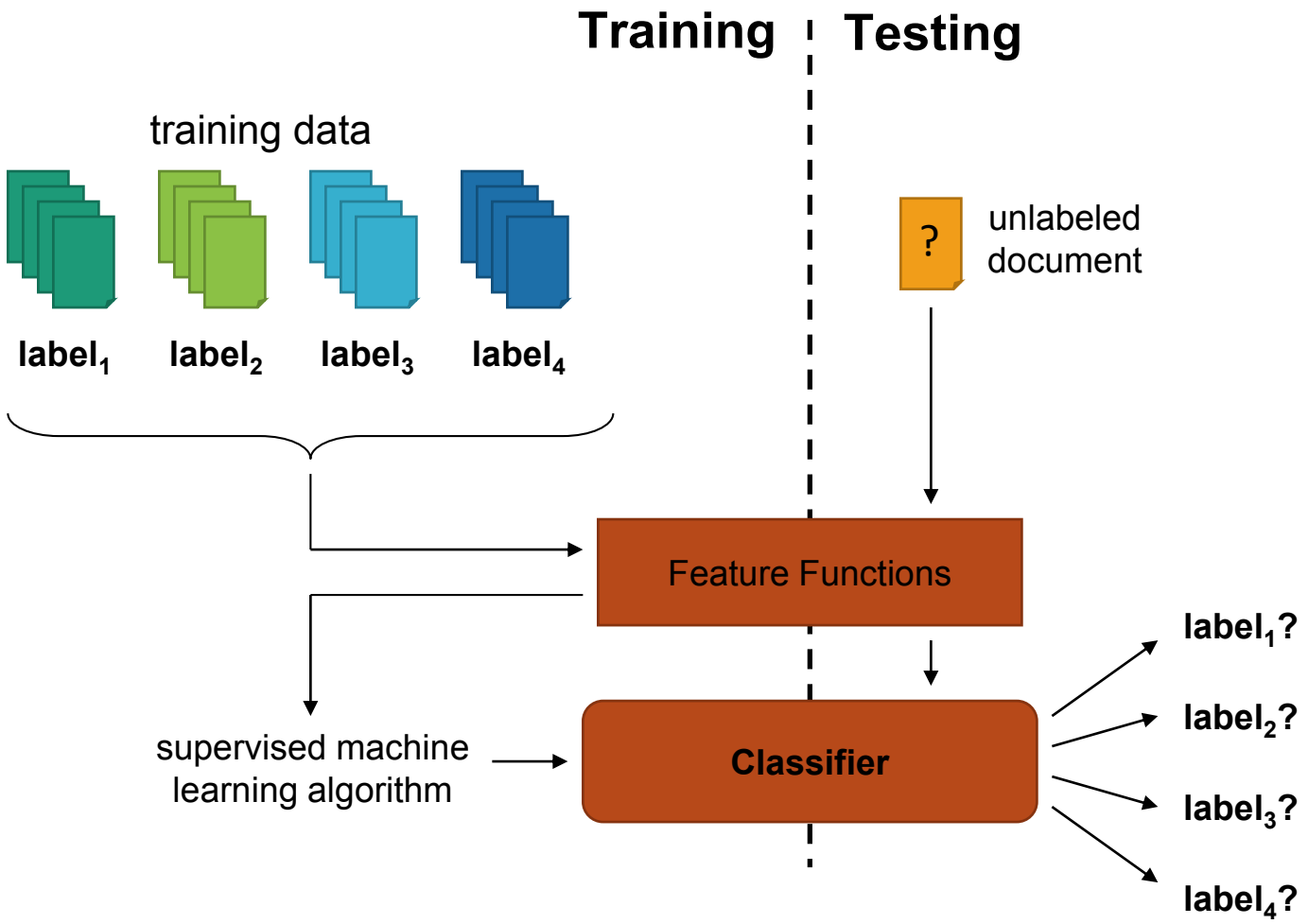
CSE4095 Introduction to NLP

Assoc. Prof. Dr. Murat Can Ganiz
Marmara University
Collage of Engineering
Computer Engineering
@BIGDaTA_Lab

Slides credit: Dan Jurafsky and mainly from Marine Carpuat

WSD as **Supervised** Classification



Evaluation Metrics for Classification

How are annotated examples used in supervised learning?

- Supervised learning = requires examples annotated with correct prediction

- Used in 2 ways:
  - To find good values for the model (hyper)parameters (training data)
  - To evaluate how good the resulting classifier is (test data)

- How do we know how good a classifier is?
  - Compare classifier predictions with human annotation
  - On held out test examples
  - Evaluation metrics: accuracy, precision, recall

Quantifying Errors in a Classification Task: The 2-by-2 contingency table (per class)

	correct	not correct
selected	tp	fp
not selected	fn	tn

Quantifying Errors in a Classification Task: Precision and Recall

	correct	not correct
selected	tp	fp
not selected	fn	tn

Precision: % of selected items that are correct

Recall: % of correct items that are selected

Q: When are Precision/Recall more informative than accuracy?

A combined measure: F

- A combined measure that assesses the P/R tradeoff is F measure (weighted harmonic mean):

$$F = \frac{1}{a\frac{1}{P} + (1 - a)\frac{1}{R}} = \frac{(b^2 + 1)PR}{b^2 P + R}$$

With $\beta^2 = \frac{1}{\alpha} - 1$

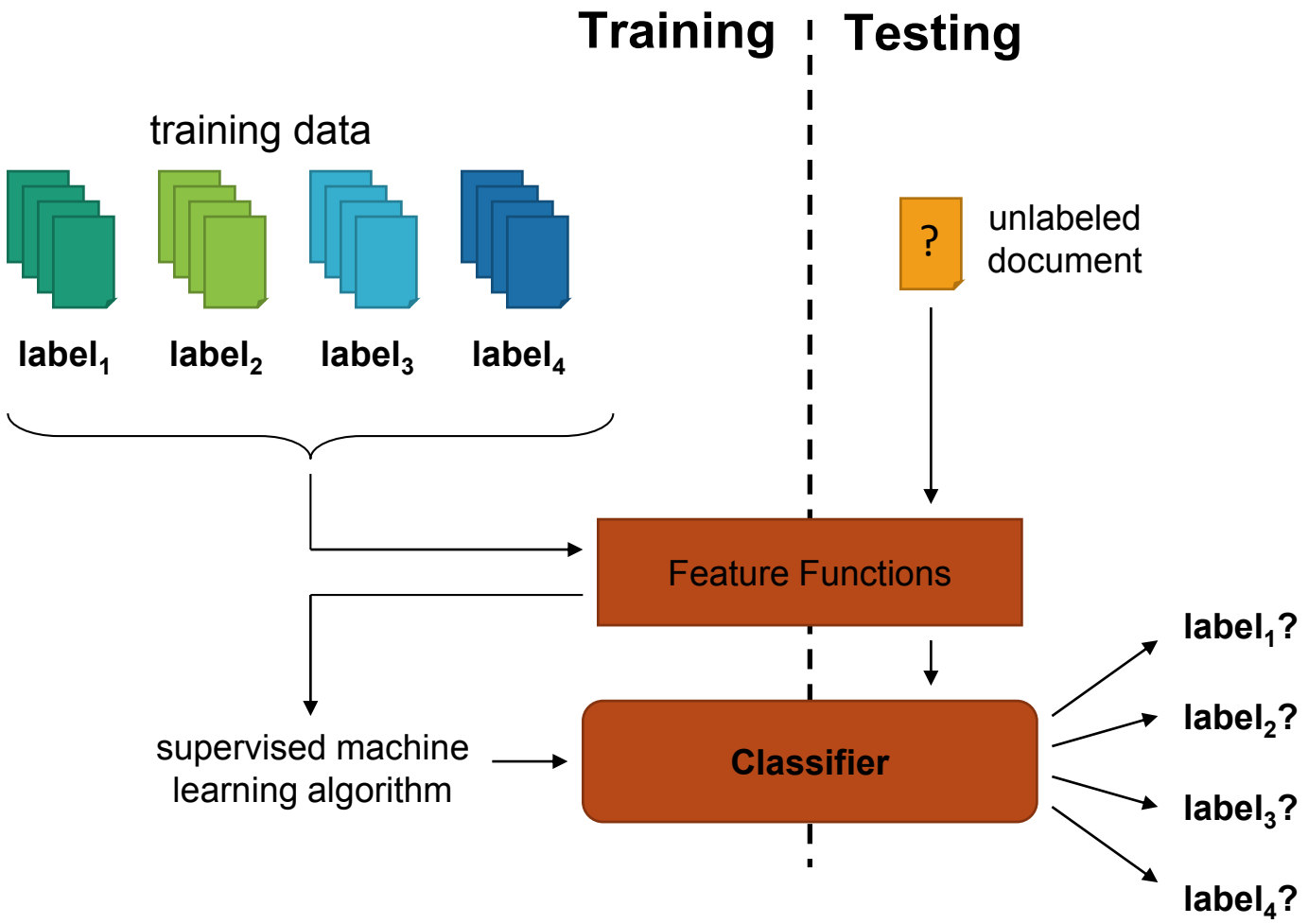
- People usually use balanced F1 measure
  - i.e., with $\beta = 1$ (that is, $\alpha = ½$):

$$F = 2PR/(P+R)$$

The Perceptron

A simple Supervised Classifier

WSD as **Supervised** Classification



Formalizing classification

Task definition

- *Given inputs*:
  - an example x

    often x is a D-dimensional vector of binary or real values
  - a fixed set of classes Y

    $Y = \{y_1, y_2, \ldots, y_J\}$

    e.g. word senses from WordNet
- *Output*: a predicted class $y \in Y$

Classifier definition

A function $f: x \rightarrow f(x) = y$

Many different types of functions/classifiers can be defined

- We'll talk about perceptron, logistic regression, neural networks.

Example:
Word Sense Disambiguation for “bass”

WordNet Sense	Spanish Translation	Roget Category	Target Word in Context
bass^4	lubina	FISH/INSECT	...fish as Pacific salmon and striped **bass** and...
bass^4	lubina	FISH/INSECT	...produce filets of smoked **bass** or sturgeon...
bass^7	bajo	MUSIC	...exciting jazz **bass** player since Ray Brown...
bass^7	bajo	MUSIC	...play **bass** because he doesn't have to solo...

- Y = {-1,+1} since there are 2 senses in our inventory

- Many different definitions of x are possible
  - E.g., vector of word frequencies for words that co-occur in a window of +/- k words around “bass”
  - Instead of frequency, we could use binary values, or tf.idf, or PPMI, etc.
  - Instead of window, we could use the entire sentence
  - Instead of/in addition to words, we could use POS tags
  - …

Perception Test Algorithm for Binary Classification: Predict class -1 or +1 for example x

$$f(x) = sign(w.x + b)$$

Algorithm 6 PERCEPTRONTEST$(w_0, w_1, \ldots, w_D, b, \hat{\boldsymbol{x}})$

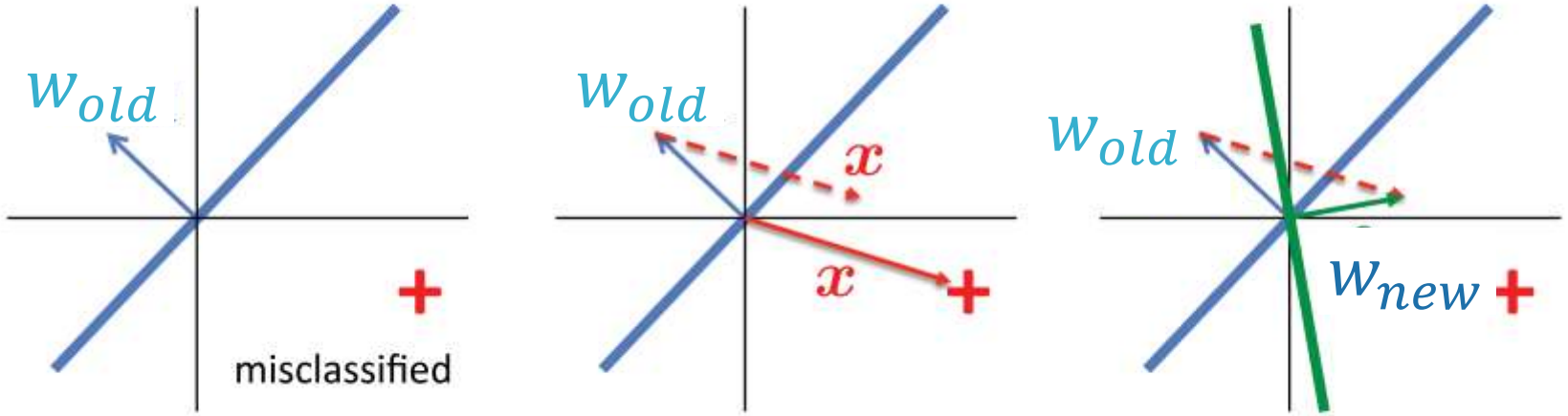
1: $a \leftarrow \sum_{d=1}^{D} w_d \, \hat{x}_d + b$ // compute activation for the test example
2: **return** SIGN(a)

Perceptron Training Algorithm:
Find good values for (w,b) given training data D

Algorithm 5 PERCEPTRONTRAIN(**D**, *MaxIter*)

```
1:  w_d ← 0, for all d = 1 ... D                          // initialize weights
2:  b ← 0                                                 // initialize bias
3:  for iter = 1 ... MaxIter do
4:      for all (x,y) ∈ D do
5:          a ← Σ_{d=1}^{D} w_d x_d + b                   // compute activation for this example
6:          if ya ≤ 0 then
7:              w_d ← w_d + y x_d, for all d = 1 ... D    // update weights
8:              b ← b + y                                 // update bias
9:          end if
10:     end for
11: end for
12: return w_0, w_1, ..., w_D, b
```

The Perceptron update rule: geometric interpretation



Machine Learning Vocabulary

x is often called the feature vector

- its elements are defined (by us, the model designers) to capture properties or features of the input that are expected to correlate with predictions

w and b are the parameters of the classifier

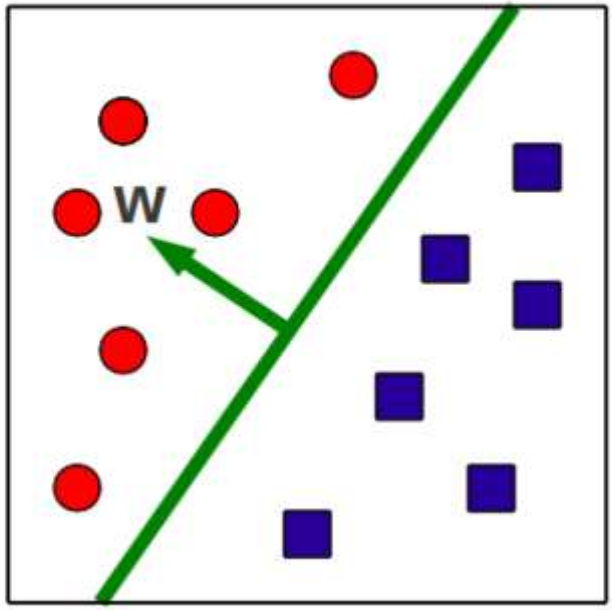
- they are needed to fully define the classification function f(x) = y
- their values are found by the training algorithm using training data D

MaxIter is a hyperparameter

- controls when training stops
- MaxIter impacts the nature of function f indirectly

All of the above affect the performance of the final classifier!

Perceptron for binary classification



- Classifier = a hyperplane that separates positive from negative examples

$$\hat{y} = sign(w.x + b)$$

- Perceptron training
  - Finds such a hyperplane
  - If training examples are separable

Convergence of Perceptron

Theorem (Block & Novikoff, 1962)

If the training data $D = \{(x_1, y_1), \ldots, (x_N, y_N)\}$ is **linearly separable** with margin γ by a unit norm hyperplane w_* ($||w_*|| = 1$) with $b = 0$,

Then **perceptron training converges after** $\frac{R^2}{\gamma^2}$ **errors** during training

(assuming ($||x|| < R$) for all x).

More Machine Learning vocabulary: overfitting/underfitting/generalization

Training error is not sufficient

- We care about **generalization** to new examples

- A classifier can classify training data perfectly, yet classify new examples incorrectly
  - Because training examples are only a sample of data distribution
    - a feature might correlate with class by coincidence
  - Because training examples could be noisy
    - e.g., accident in labeling

Overfitting

- Consider a model θ and its:
  - Error rate over training data $error_{train}(\theta)$
  - True error rate over all data $error_{true}(\theta)$

- We say h overfits the training data if

$$error_{train}(\theta) < error_{true}(\theta)$$

Evaluating on test data

- Problem: we don't know $error_{true}(\theta)$!

- Solution:
  - we set aside a test set
    - some examples that will be used for evaluation
  - we don't look at them during training!
  - after learning a classifier θ, we calculate $error_{test}(\theta)$

Overfitting

- Another way of putting it

- A classifier θ is said to overfit the training data, if there are other parameters θ', such that
  - θ has a smaller error than θ' on the training data
  - but θ has larger error on the test data than θ'.

Underfitting/Overfitting

- Underfitting
  - Learning algorithm had the opportunity to learn more from training data, but didn’t
- Overfitting
  - Learning algorithm paid too much attention to idiosyncracies of the training data; the resulting classifier doesn’t generalize

Back to the Perceptron

- Practical strategies to improve generalization for the perceptron
  - Voting/Averaging
  - Randomize order of training data
  - Use a development test set to find good hyperparameter values
    - E.g., early stopping is a good strategy to avoid overfitting

The Perceptron
What you should know

- What is the underlying function used to make predictions
- Perceptron test algorithm
- Perceptron training algorithm
- How to improve perceptron training with the averaged perceptron
- Fundamental Machine Learning Concepts:
  - train vs. test data; parameter; hyperparameter; generalization; overfitting; underfitting.
- How to define features

Logistic Regression for **Binary** Classification

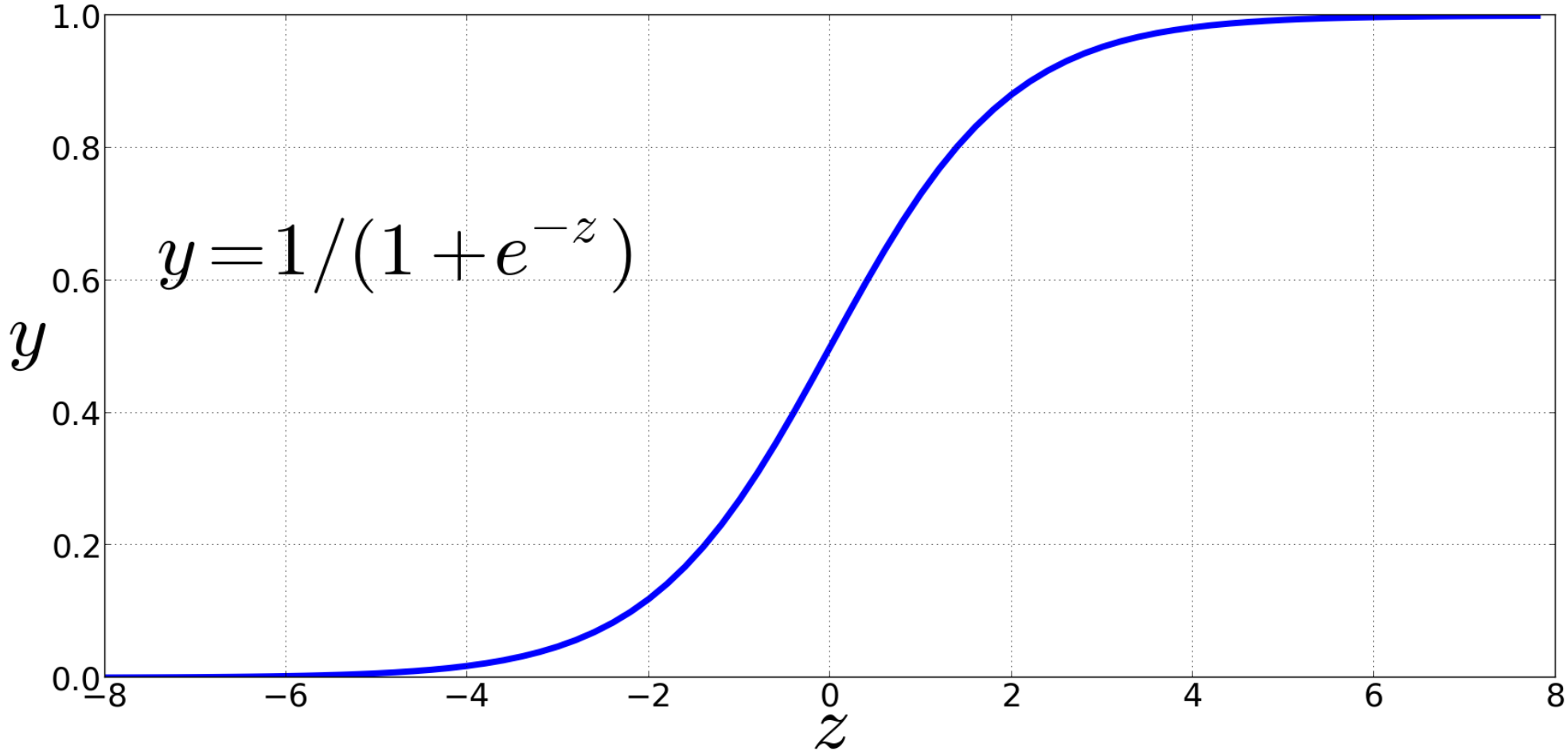
Images and examples: Jurafsky & Martin, SLP 3 Chapter 5

From Perceptron to Probabilities: the Logistic Regression classifier

- The perceptron gives us a prediction y, and the activation can take any real value

- What if we want a probability p(y|x) instead?

The sigmoid function (aka the logistic function)



From Perceptron to Probabilities for Binary Classification

$$
\begin{aligned}
P(y=1) &= \sigma(w \cdot x + b) \\
&= \frac{1}{1+e^{-(w \cdot x + b)}} \\
P(y=0) &= 1 - \sigma(w \cdot x + b) \\
&= 1 - \frac{1}{1+e^{-(w \cdot x + b)}} \\
&= \frac{e^{-(w \cdot x + b)}}{1+e^{-(w \cdot x + b)}}
\end{aligned}
$$

Making Predictions with the Logistic Regression Classifier

- Given a test instance x, predict class 1 if P(y=1|x) > 0.5, and 0 otherwise

$$\hat{y} = \begin{cases} 1 & \text{if } P(y=1|x) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

- Inputs x for which P(y=1|x) = 0.5 constitute the **decision boundary**

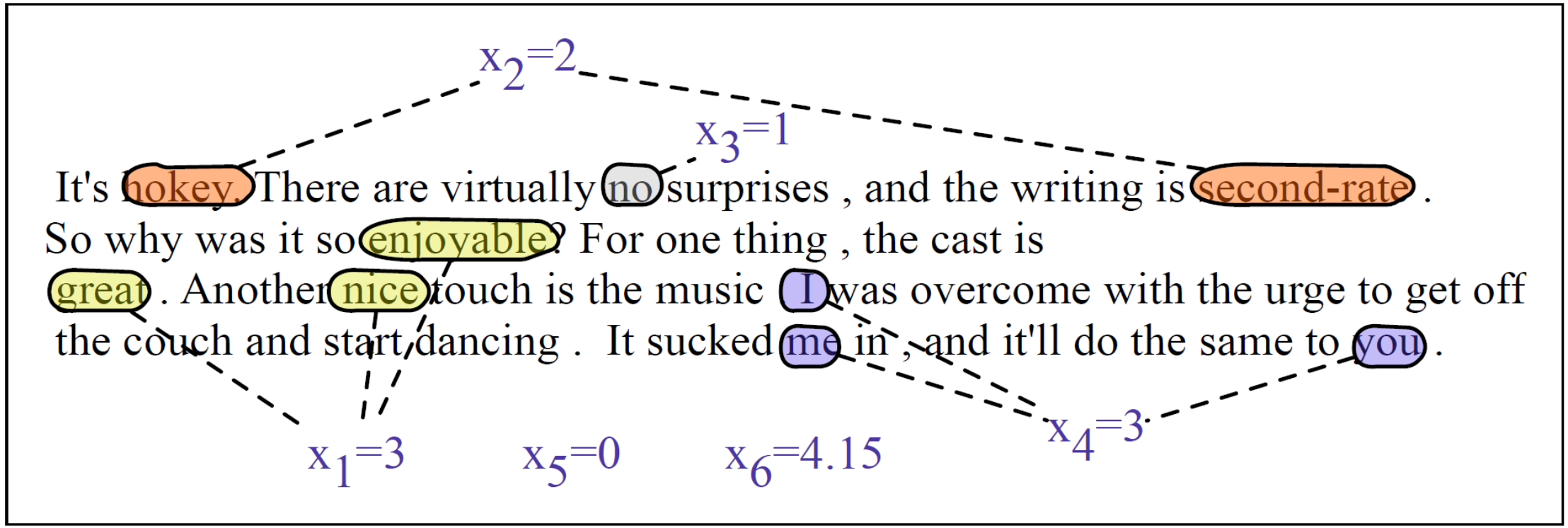
Example: Sentiment Classification with Logistic Regression

- 2 classes: 1 (positive sentiment) or 0 (negative sentiment)
- Examples are movie reviews
- Features:

Var	Definition	Value in Fig. 5.2
x_1	count(positive lexicon) $\in$ doc)	3
x_2	count(negative lexicon) $\in$ doc)	2
x_3	$\begin{cases} 1 & \text{if ``no'' } \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	1
x_4	count(1st and 2nd pronouns $\in$ doc)	3
x_5	$\begin{cases} 1 & \text{if ``!'' } \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	0
x_6	log(word count of doc)	$\ln(64) = 4.15$

Constructing the feature vector x for one example

Var	Definition	Value in Fig. 5.2
x_1	count(positive lexicon) $\in$ doc)	3
x_2	count(negative lexicon) $\in$ doc)	2
x_3	$\begin{cases} 1 & \text{if "no"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	1
x_4	count(1st and 2nd pronouns $\in$ doc)	3
x_5	$\begin{cases} 1 & \text{if "!"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	0
x_6	log(word count of doc)	$\ln(64) = 4.15$



Example: Sentiment Classification with Logistic Regression

- Assume we are given the parameters of the classifier

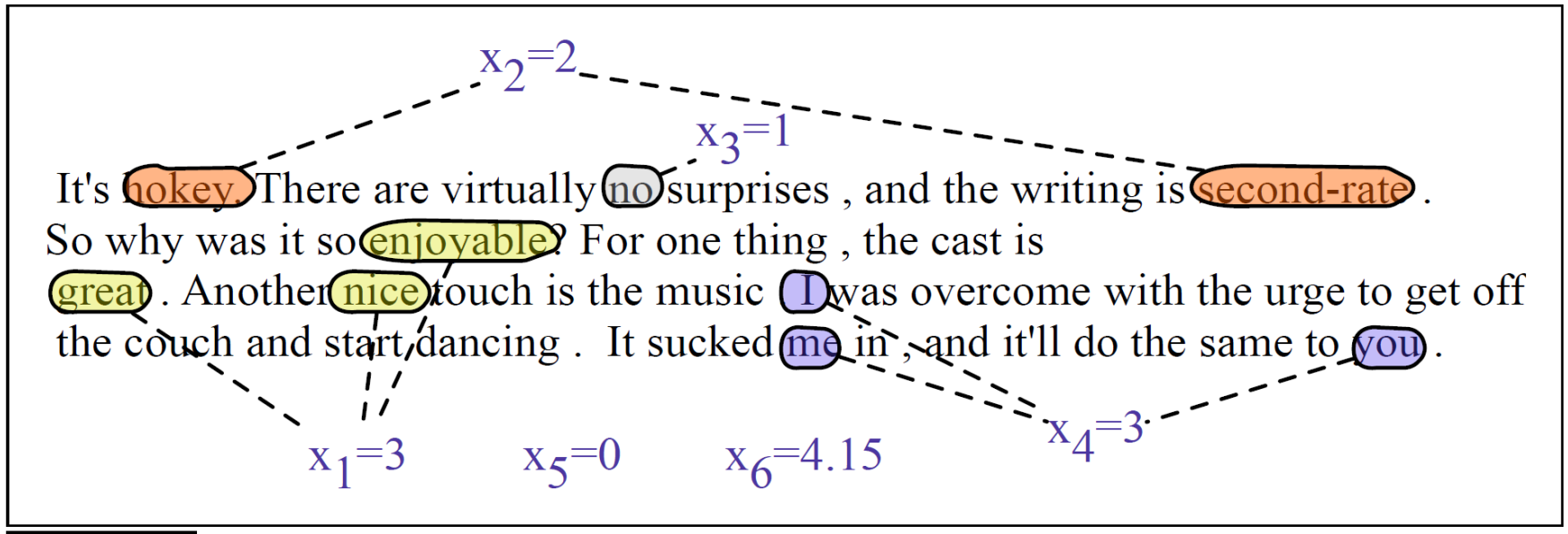
w = $[2.5, -5.0, -1.2, 0.5, 2.0, 0.7]$

b = 0.1

- On this example:

P(y=1|x) = 0.69

P(y=0|x) = 0.31



Learning in Logistic Regression

- How are parameters of the model (w and b) learned?

- This is an instance of supervised learning
  - We have labeled training examples

- We want model parameters such that
  - For training examples x
  - The prediction of the model $\hat{y}$
  - is as close as possible to the true y

Learning in Logistic Regression

- How are parameters of the model (w and b) learned?

- This is an instance of supervised learning
  - We have labeled training examples

- We want model parameters such that
  - For training examples x, the prediction of the model $\hat{y}$ is as close as possible to the true y
  - Or equivalently so that the distance between $\hat{y}$ and y is small

Ingredients required for training

- Loss function or cost function
  - A measure of distance between classifier prediction and true label for a given set of parameters

$$L(\hat{y}, y) = \text{How much } \hat{y} \text{ differs from the true } y$$

- An algorithm to minimize this loss
  - Here we'll introduce stochastic gradient descent

The cross-entropy loss function

- Loss function used for logistic regression and often for neural networks

- Defined as follows:

$$L_{CE}(w,b) = -[y \log \sigma(w \cdot x + b) + (1-y) \log (1 - \sigma(w \cdot x + b))]$$

Deriving the cross-entropy loss function

- Conditional maximum likelihood
  - Choose parameters that maximize the log probability of true labels y given inputs x

$$\begin{aligned}\log p(y|x) &= \log\left[\hat{y}^{y}\left(1-\hat{y}\right)^{1-y}\right] \\ &= y\log\hat{y}+(1-y)\log(1-\hat{y})\end{aligned}$$

- Cross-entropy loss is defined as

$$L_{CE}(\hat{y},y) = -\log p(y|x) = -\left[y\log\hat{y}+(1-y)\log(1-\hat{y})\right]$$

Example: Sentiment Classification with Logistic Regression

- Assume we are given the parameters of the classifier

w = $[2.5, -5.0, -1.2, 0.5, 2.0, 0.7]$

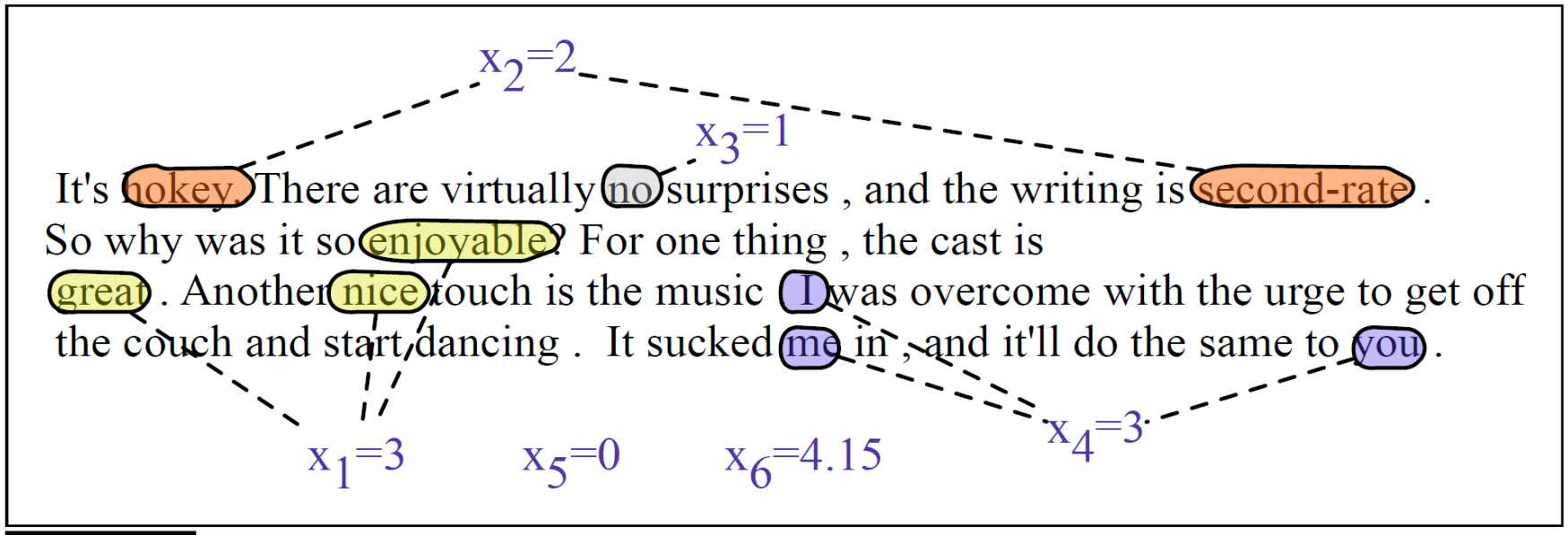
b = 0.1

- On this example:

P(y=1|x) = 0.69

P(y=0|x) = 0.31

Loss(w,b) = - log(0.69) = 0.37



Example: Sentiment Classification with Logistic Regression

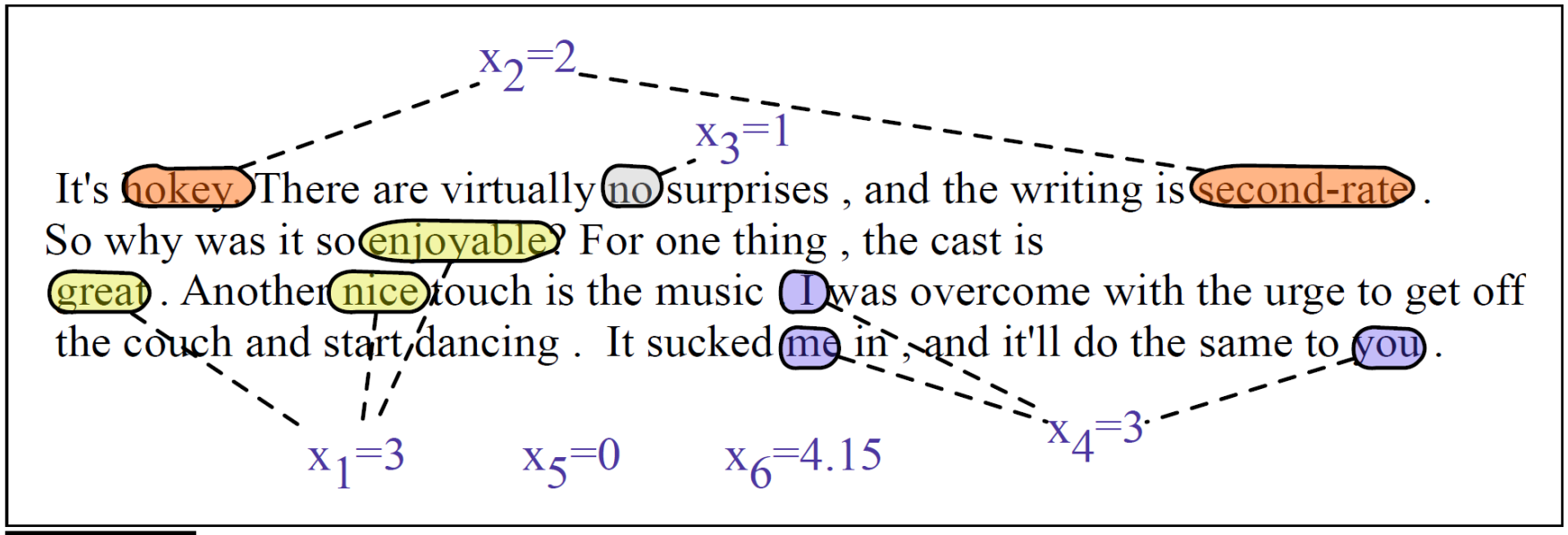
- Assume we are given the parameters of the classifier

w = $[2.5, -5.0, -1.2, 0.5, 2.0, 0.7]$

b = 0.1

- If the example was negative (y=0)

Loss(w,b) = - log(0.31) = 1.17



Gradient Descent

- Goal:
  - find parameters $\theta = w, b$
  - Such that

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} \frac{1}{m} \sum_{i=1}^{m} L_{CE}(y^{(i)}, x^{(i)}; \theta)$$

- For logistic regression, the loss is **convex**

Illustrating Gradient Descent

The **gradient** indicates the direction of greatest increase of the cost/loss function.

Gradient descent finds parameters (w,b) that decrease the loss by taking a step in the opposite direction of the gradient.

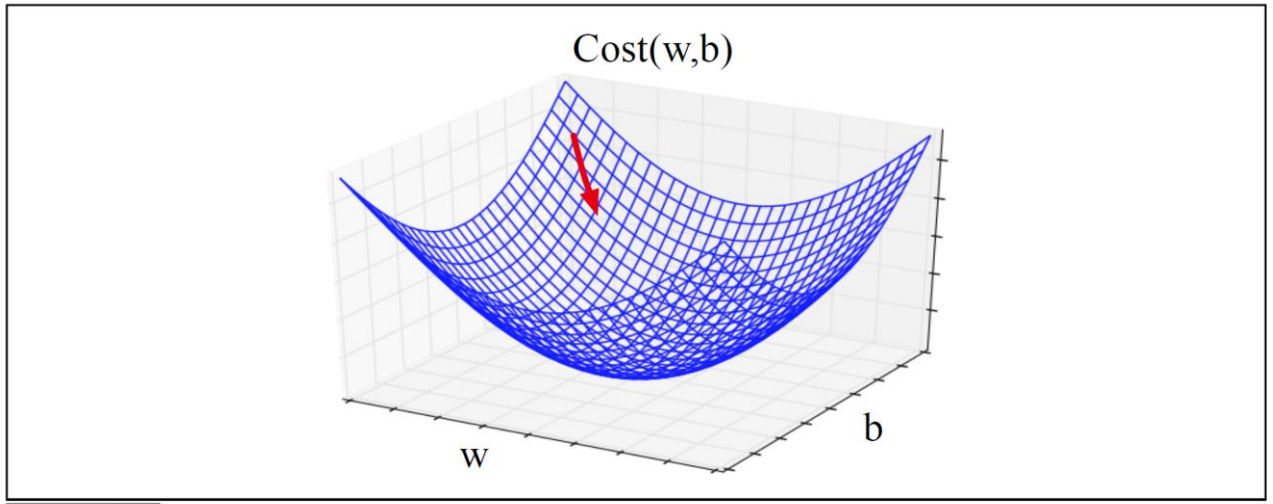


Figure 5.4 Visualization of the gradient vector in two dimensions *w* and *b*.

function STOCHASTIC GRADIENT DESCENT($L()$, $f()$, x, y) **returns** θ

where: L is the loss function
f is a function parameterized by θ
x is the set of training inputs $x^{(1)}$, $x^{(2)}$,..., $x^{(n)}$
y is the set of training outputs (labels) $y^{(1)}$, $y^{(2)}$,..., $y^{(n)}$

$\theta \leftarrow 0$
repeat til done # see caption
  For each training tuple $(x^{(i)}, y^{(i)})$ (in random order)
    1. Optional (for reporting): # How are we doing on this tuple?
      Compute $\hat{y}^{(i)} = f(x^{(i)}; \theta)$ # What is our estimated output $\hat{y}$?
      Compute the loss $L(\hat{y}^{(i)}, y^{(i)})$ # How far off is $\hat{y}^{(i)}$) from the true output $y^{(i)}$?
    2. $g \leftarrow \nabla_\theta L(f(x^{(i)}; \theta), y^{(i)})$ # How should we move θ to maximize loss?
    3. $\theta \leftarrow \theta - \eta\, g$ # Go the other way instead
return θ

Figure 5.5 The stochastic gradient descent algorithm. Step 1 (computing the loss) is used to report how well we are doing on the current tuple. The algorithm can terminate when it converges (or when the gradient $< \varepsilon$), or when progress halts (for example when the loss starts going up on a held-out set).

The gradient for logistic regression

$$L_{CE}(w,b) = -[y \log \sigma(w \cdot x + b) + (1-y) \log (1 - \sigma(w \cdot x + b))]$$

$$\frac{\partial L_{CE}(w,b)}{\partial w_j} = [\sigma(w \cdot x + b) - y] x_j$$

Difference between the model prediction and the correct answer y

Feature value for dimension j

Note: the detailed derivation is available in the reading (SLP3 Chapter 5, section 5.8)

Logistic Regression
What you should know

How to make a prediction with logistic regression classifier

How to train a logistic regression classifier

Machine learning concepts:

Loss function

Gradient Descent Algorithm

function STOCHASTIC GRADIENT DESCENT($L()$, $f()$, x, y) **returns** θ

where: L is the loss function
f is a function parameterized by θ
x is the set of training inputs $x^{(1)}$, $x^{(2)}$,..., $x^{(n)}$
y is the set of training outputs (labels) $y^{(1)}$, $y^{(2)}$,..., $y^{(n)}$

$\theta \leftarrow 0$
repeat til done # see caption
  For each training tuple $(x^{(i)}, y^{(i)})$ (in random order)
    1. Optional (for reporting): # How are we doing on this tuple?
      Compute $\hat{y}^{(i)} = f(x^{(i)}; \theta)$ # What is our estimated output $\hat{y}$?
      Compute the loss $L(\hat{y}^{(i)}, y^{(i)})$ # How far off is $\hat{y}^{(i)}$) from the true output $y^{(i)}$?
    2. $g \leftarrow \nabla_\theta L(f(x^{(i)}; \theta), y^{(i)})$ # How should we move θ to maximize loss?
    3. $\theta \leftarrow \theta - \eta\, g$ # Go the other way instead
return θ

Figure 5.5 The stochastic gradient descent algorithm. Step 1 (computing the loss) is used to report how well we are doing on the current tuple. The algorithm can terminate when it converges (or when the gradient $< \varepsilon$), or when progress halts (for example when the loss starts going up on a held-out set).

SGD hyperparameter: the learning rate

- The hyperparameter η that control the size of the step down the gradient is called the learning rate
- If η is too large, training might not converge; if η is too small, training might be very slow.
- How to set the learning rate? Common strategies:
  - decay over time: $\eta = \frac{1}{C+t}$



  - Use held-out test set, increase learning rate when likelihood increases

Multiclass Logistic Regression

Formalizing classification

Task definition

- *Given inputs*:
  - an example x

    often x is a D-dimensional vector of binary or real values
  - a fixed set of classes Y

    $Y = \{y_1, y_2, \ldots, y_J\}$

    e.g. word senses from WordNet
- *Output*: a predicted class $y \in Y$

Classifier definition

A function $g: x \rightarrow g(x) = y$

Many different types of functions/classifiers can be defined

- We'll talk about perceptron, logistic regression, neural networks.

So far we've only worked with binary classification problems i.e. J = 2

A multiclass logistic regression classifier

aka multinomial logistic regression, softmax logistic regression, maximum entropy (or maxent) classifier

Goal: predict probability $P(y=c|x)$, where c is one of k classes in set C

The softmax function

- A generalization of the sigmoid
- Input: a vector z of dimensionality k

$$z = [z_1, z_2, \ldots, z_k]$$

- Output: a vector of dimensionality k

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}} \quad 1 \leq i \leq k$$

$$\text{softmax}(z) = \left[\frac{e^{z_1}}{\sum_{i=1}^{k} e^{z_i}}, \frac{e^{z_2}}{\sum_{i=1}^{k} e^{z_i}}, \ldots, \frac{e^{z_k}}{\sum_{i=1}^{k} e^{z_i}} \right]$$

Looks like a probability distribution!

The softmax function
Example

Thus for example given a vector:

$$z = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1]$$

the result softmax(z) is

$$[0.055, 0.090, 0.0067, 0.10, 0.74, 0.010]$$

All values are in [0,1] and sum up to 1: they can be interpreted as probabilities!

A multiclass logistic regression classifier

aka multinomial logistic regression, softmax logistic regression, maximum entropy (or maxent) classifier

Goal: predict probability P(y=c|x), where c is one of k classes in set C

Model definition:

$$p(y=c|x) = \frac{e^{w_c \cdot x + b_c}}{\sum_{j=1}^{k} e^{w_j \cdot x + b_j}}$$

We now have one weight vector and one bias PER CLASS

Features in multiclass logistic regression

- Features are a function of the input example **and of a candidate output class c**
- $f_i(c, x)$ represents feature i for a particular class c for a given example x

Example: sentiment analysis with 3 classes {positive (+), negative (-), neutral (0)}

- Starting from the features for binary classification
- We create one copy of each feature per class

Var	Definition	Wt
$f_1(0,x)$	1 if "!" $\in$ doc 0 otherwise	-4.5
$f_1(+,x)$	1 if "!" $\in$ doc 0 otherwise	2.6
$f_1(-,x)$	1 if "!" $\in$ doc 0 otherwise	1.3

Learning in Multiclass Logistic Regression

- Loss function for a single example

$$L_{CE}(\hat{y}, y) = -\sum_{k=1}^{K} 1\{y = k\} \log p(y = k|x)$$

1{ } is an indicator function that evaluates to 1 if the condition in the brackets is true, and to 0 otherwise

Learning in Multiclass Logistic Regression

- Loss function for a single example

$$
\begin{aligned}
L_{CE}(\hat{y}, y) &= -\sum_{k=1}^{K} 1\{y = k\} \log p(y = k|x) \\
&= -\sum_{k=1}^{K} 1\{y = k\} \log \frac{e^{w_k \cdot x + b_k}}{\sum_{j=1}^{K} e^{w_j \cdot x + b_j}}
\end{aligned}
$$

Learning in Multiclass Logistic Regression

$$
\begin{aligned}
L_{CE}(\hat{y}, y) &= -\sum_{k=1}^{K} 1\{y = k\} \log p(y = k|x) \\
&= -\sum_{k=1}^{K} 1\{y = k\} \log \frac{e^{w_k \cdot x + b_k}}{\sum_{j=1}^{K} e^{w_j \cdot x + b_j}}
\end{aligned}
$$

$$
\begin{aligned}
\frac{\partial L_{CE}}{\partial w_k} &= -(1\{y = k\} - p(y = k|x))x_k \\
&= -\left(1\{y = k\} - \frac{e^{w_k \cdot x + b_k}}{\sum_{j=1}^{K} e^{w_j \cdot x + b_j}}\right) x_k
\end{aligned}
$$

Logistic Regression
What you should know

How to make a prediction with logistic regression classifier

How to train a logistic regression classifier

For both binary and multiclass problems

Machine learning concepts:

Loss function

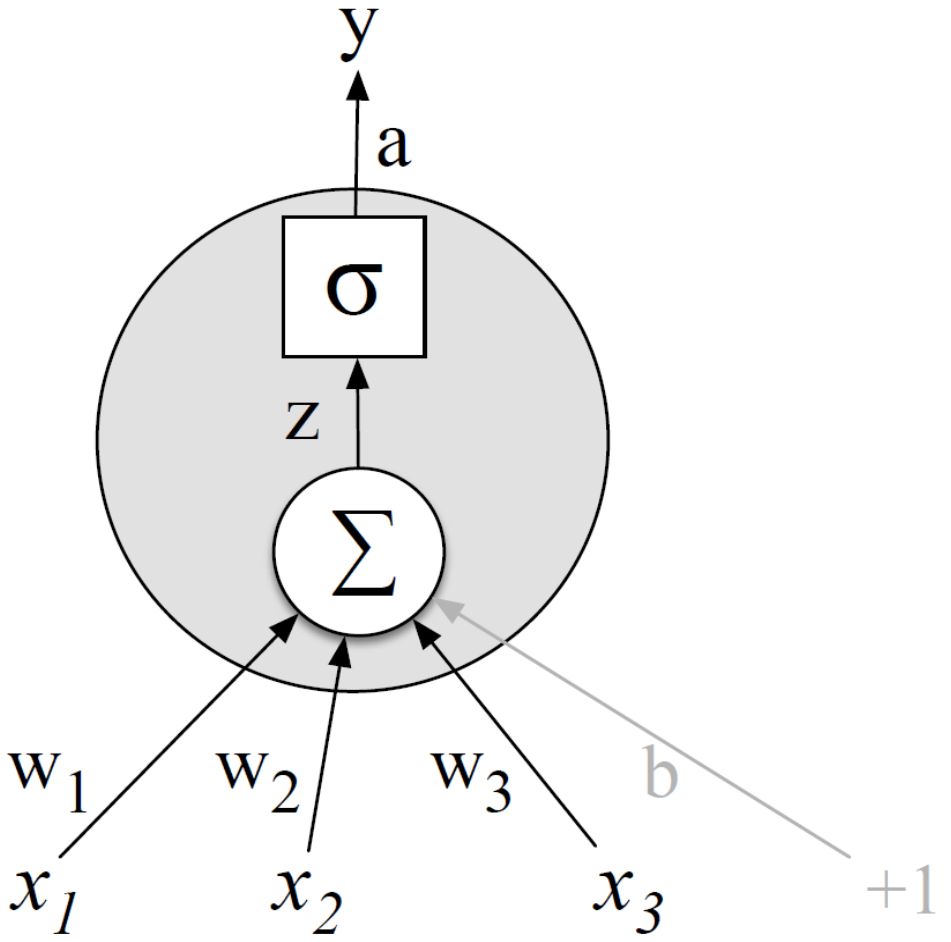
Gradient Descent Algorithm

Learning rate

Neural Networks

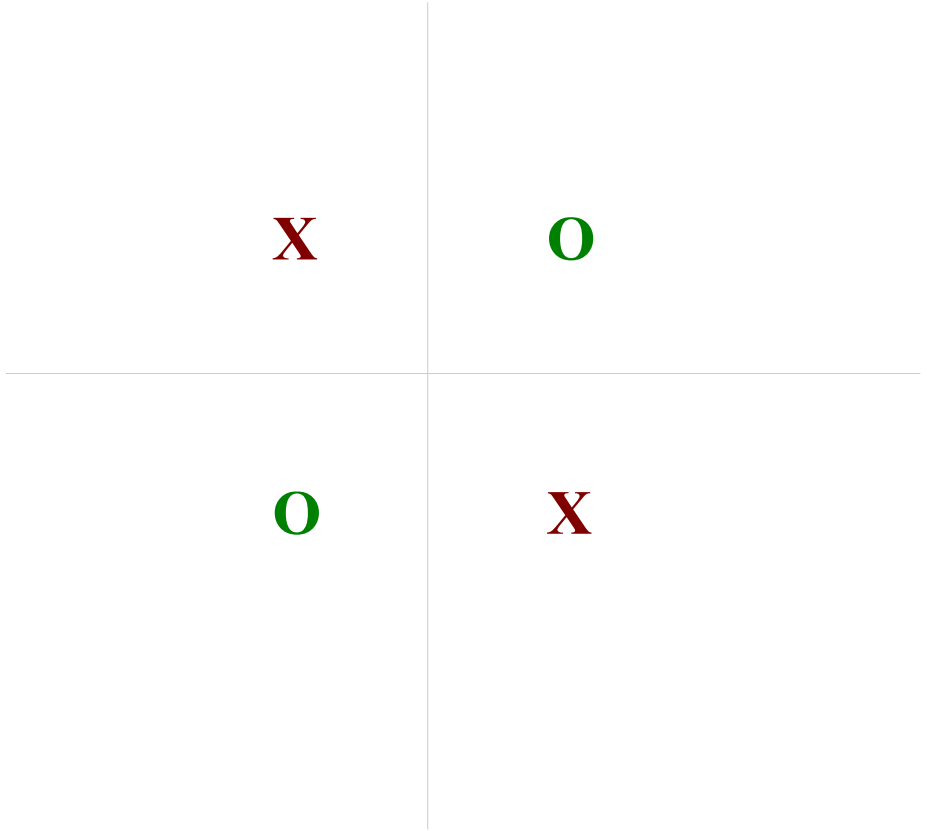
From logistic regression to a neural network unit

$$y = \sigma(w \cdot x + b) = \frac{1}{1 + \exp(-(w \cdot x + b))}$$



Limitation of perceptron

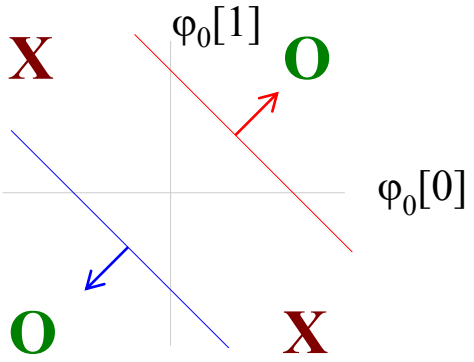
- can only find **linear separations** between positive and negative examples



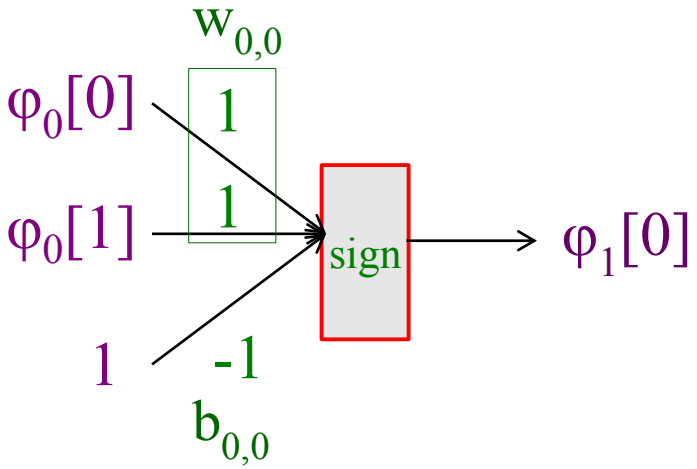
Example: binary classification with a neural network

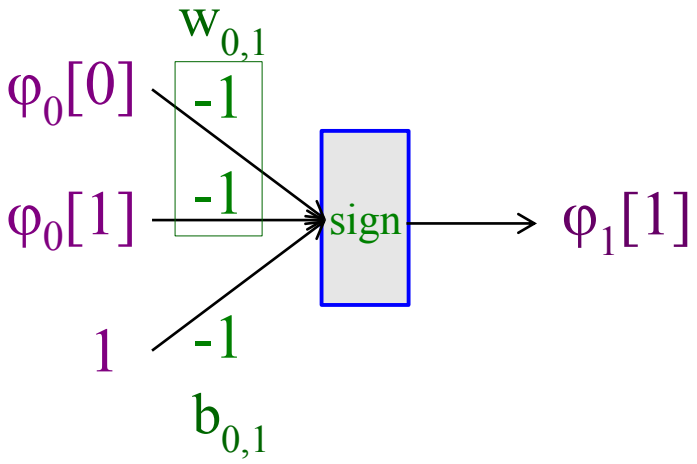
- Create two classifiers

$\varphi_0(x_1) = \{-1, 1\}$ $\varphi_0(x_2) = \{1, 1\}$



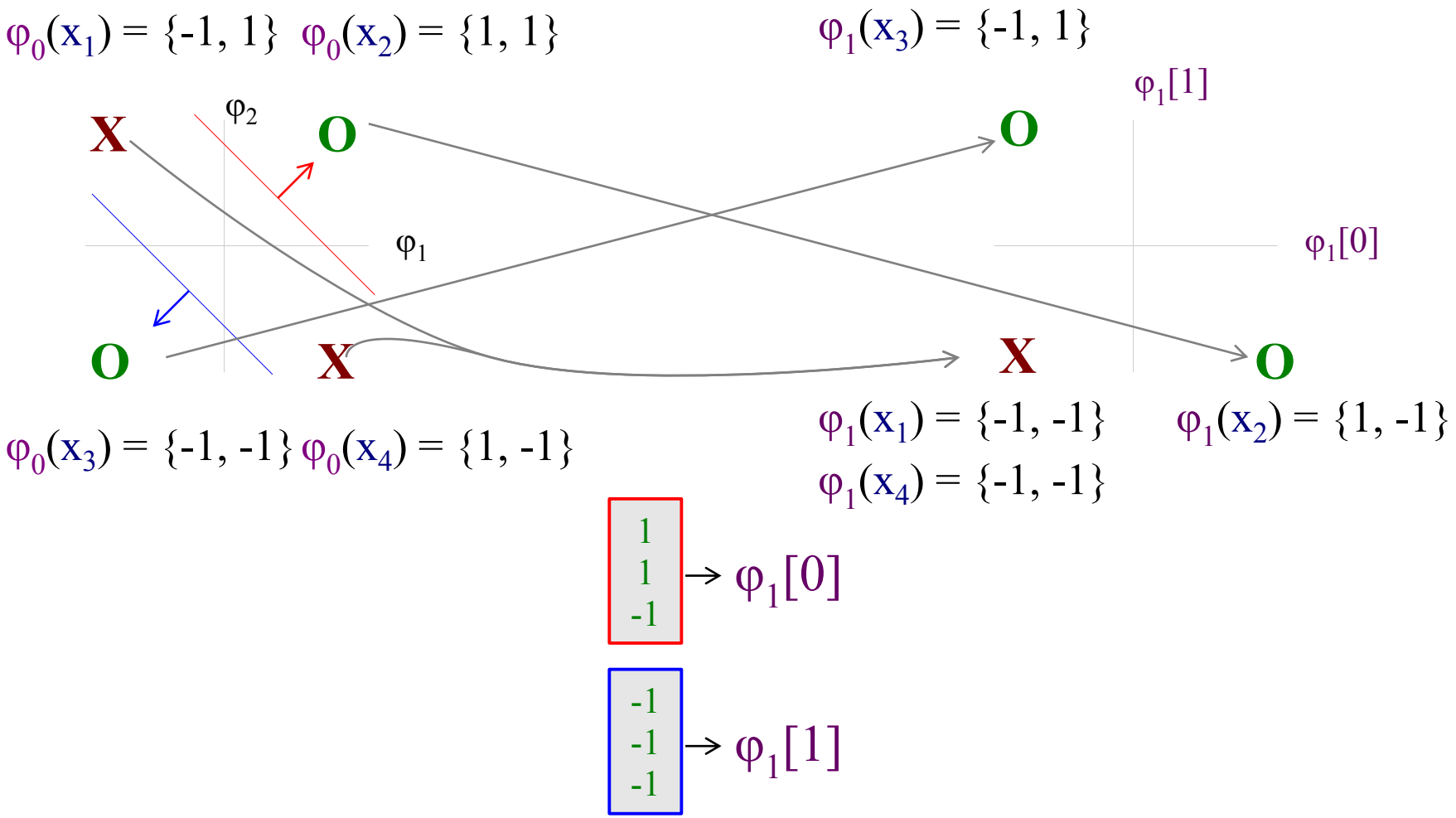
$\varphi_0(x_3) = \{-1, -1\}$ $\varphi_0(x_4) = \{1, -1\}$



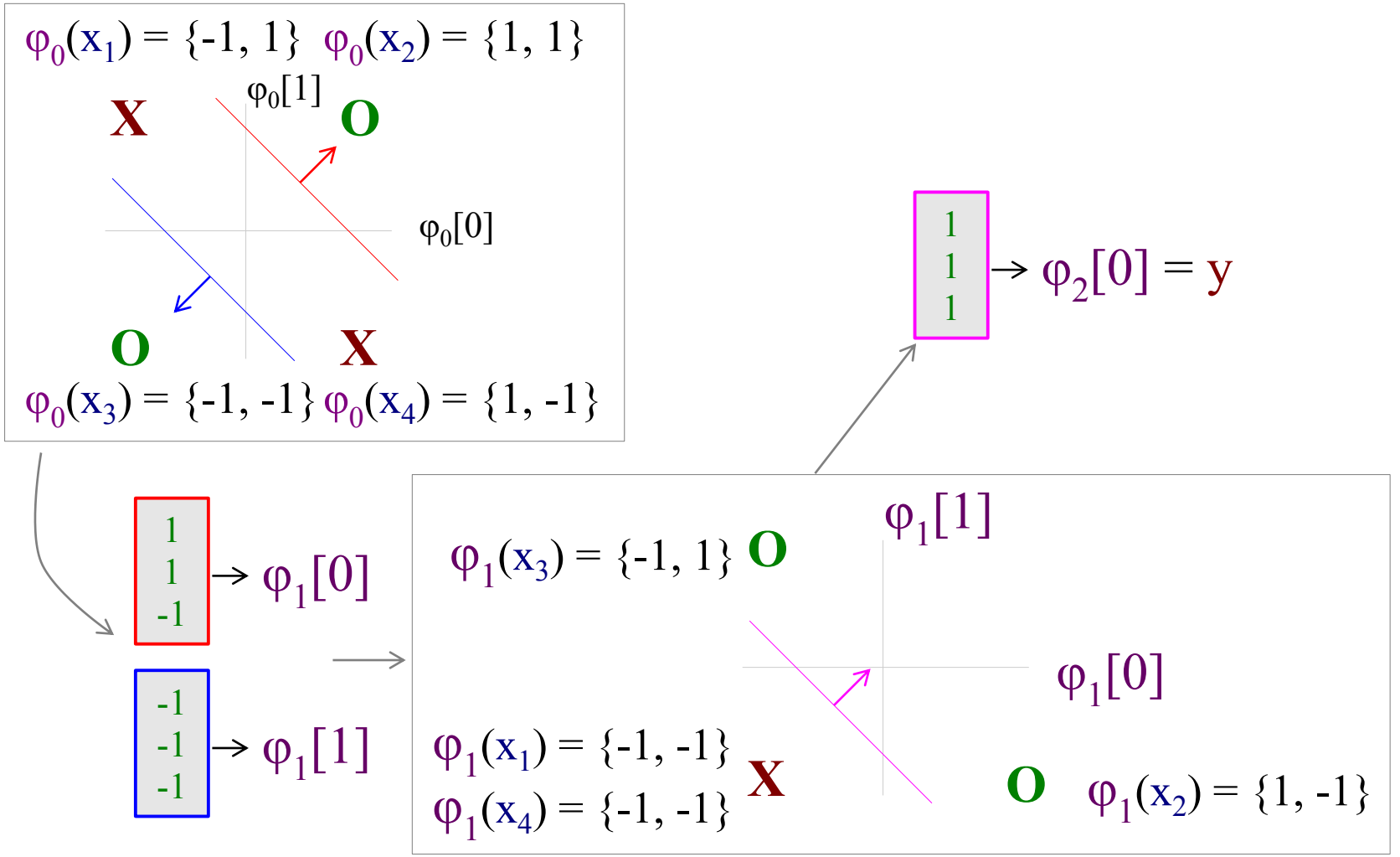


Example: binary classification with a neural network

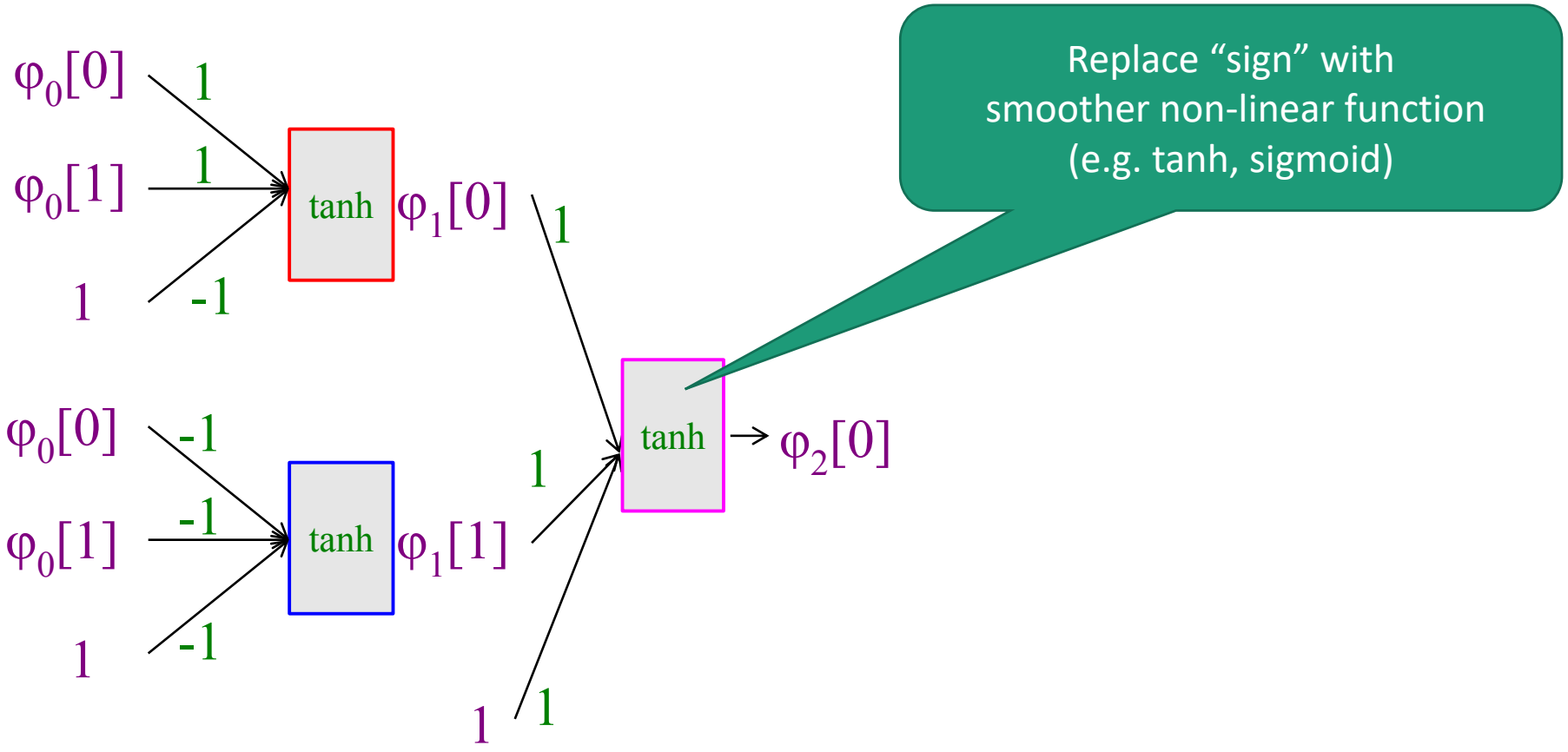
- These classifiers map to a new space



Example: binary classification with a neural network



Example: the final network can correctly classify the examples that the perceptron could not.



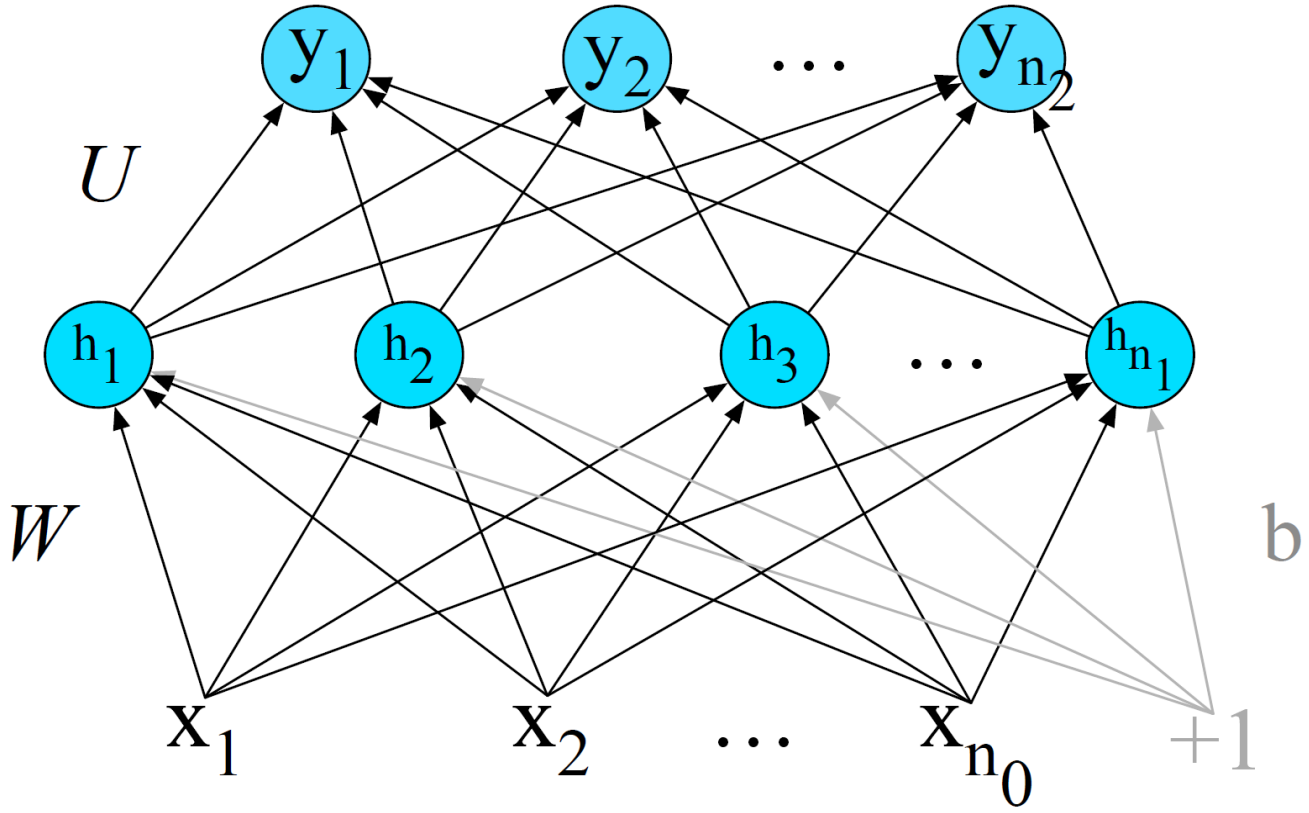
Feedforward Neural Networks

Components:

- an input layer
- an output layer
- one or more hidden layers

In a fully connected network:
each hidden unit takes as input
all the units in the previous laye

No loops!



A 2-layer feedforward neural network

Designing Neural Networks: Activation functions

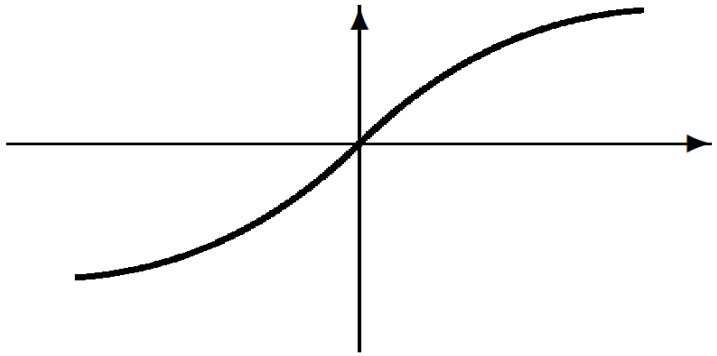
- Hidden layer can be viewed as set of hidden features
- The output of the hidden layer indicates the extent to which each hidden feature is “activated” by a given input
- The activation function is a non-linear function that determines range of hidden feature values

Designing Neural Networks: Activation functions

Hyperbolic tangent

$$\tanh(x) = \frac{\sinh(x)}{\cosh(x)} = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

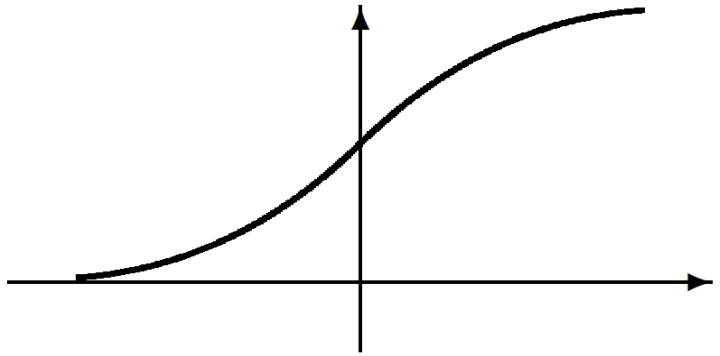
output ranges
from –1 to +1



Logistic function

$$\text{sigmoid}(x) = \frac{1}{1+e^{-x}}$$

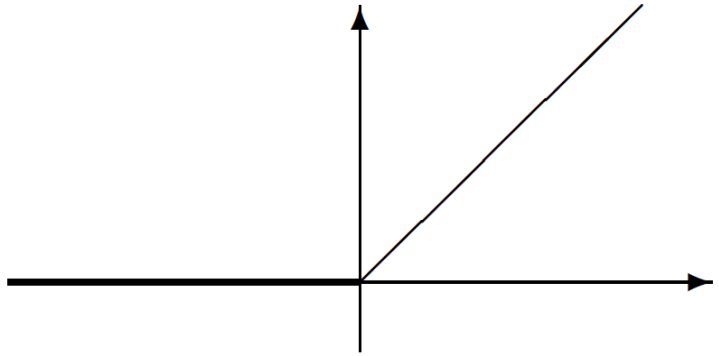
output ranges
from 0 to +1



Rectified linear unit

$$\text{relu}(x) = \max(0,x)$$

output ranges
from 0 to ∞

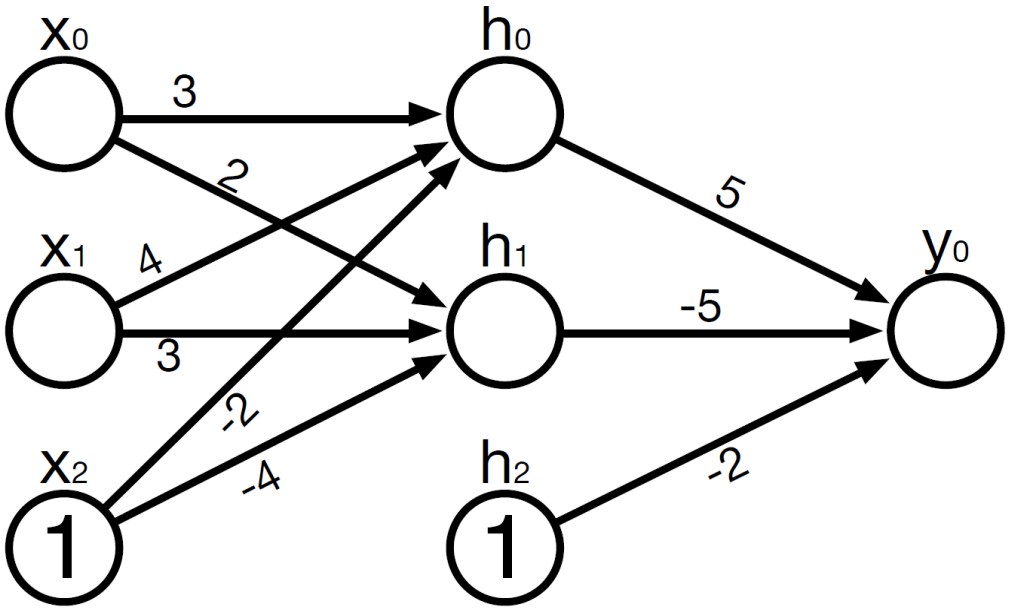


Designing Neural Networks: Network structure

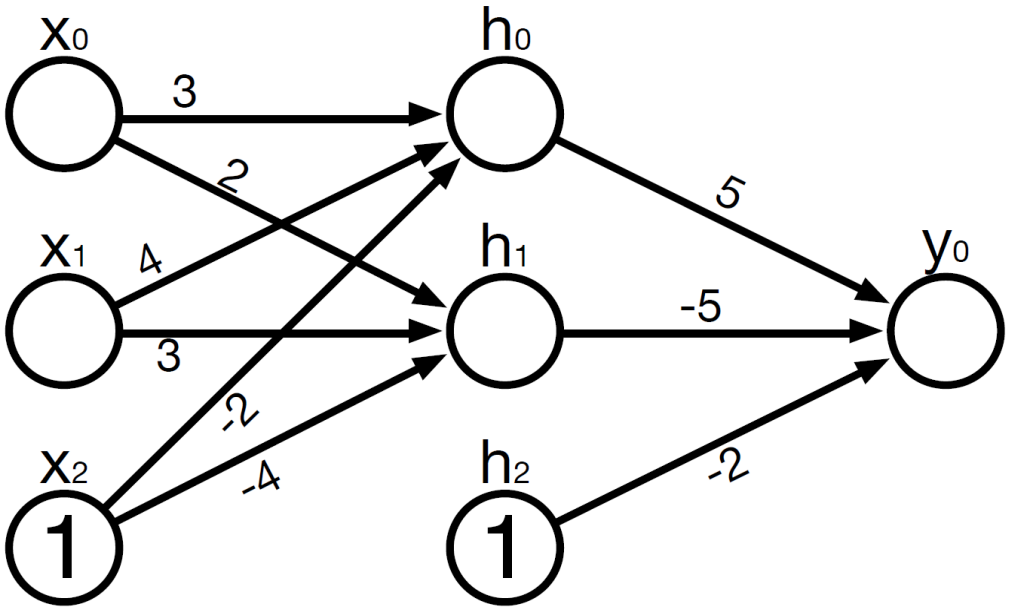
- 2 key decisions:
  - Width (number of nodes per layer)
  - Depth (number of hidden layers)

- More parameters means that the network can learn more complex functions of the input

Forward Propagation: For a given network, and some input values, compute output



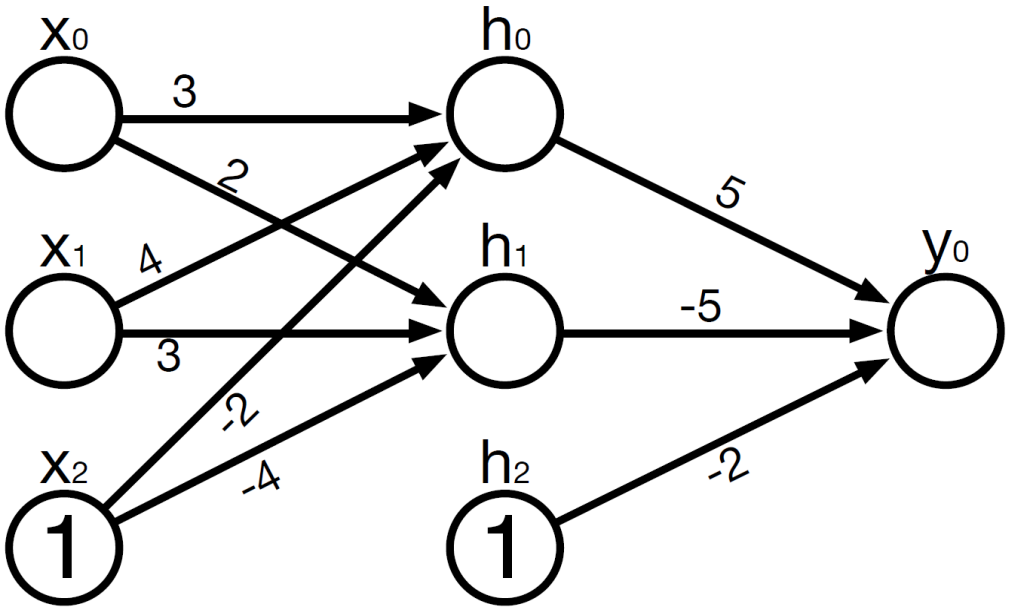
Forward Propagation: For a given network, and some input values, compute output



Given input (1,0) (and sigmoid non-linearities), we can calculate the output by processing one layer at a time:

Layer	Node	Summation	Activation
hidden	h_0	$1 \times 3 + 0 \times 4 + 1 \times -2 = 1$	0.731
hidden	h_1	$1 \times 2 + 0 \times 3 + 1 \times -4 = -2$	0.119
output	y_0	$0.731 \times 5 + 0.119 \times -5 + 1 \times -2 = 1.060$	0.743

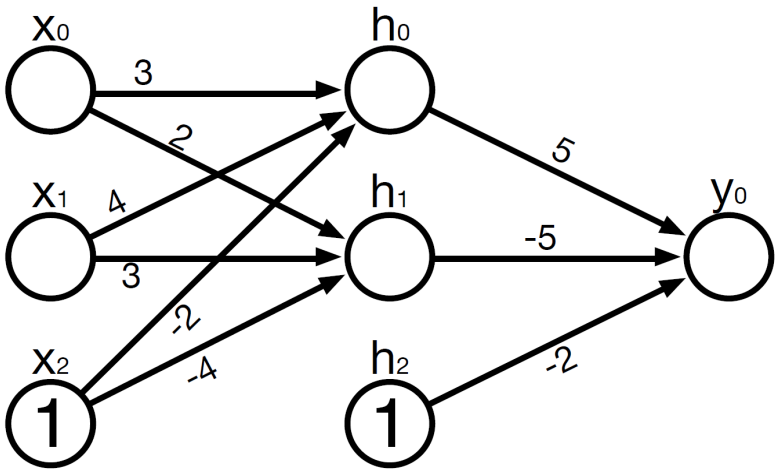
Forward Propagation: For a given network, and some input values, compute output

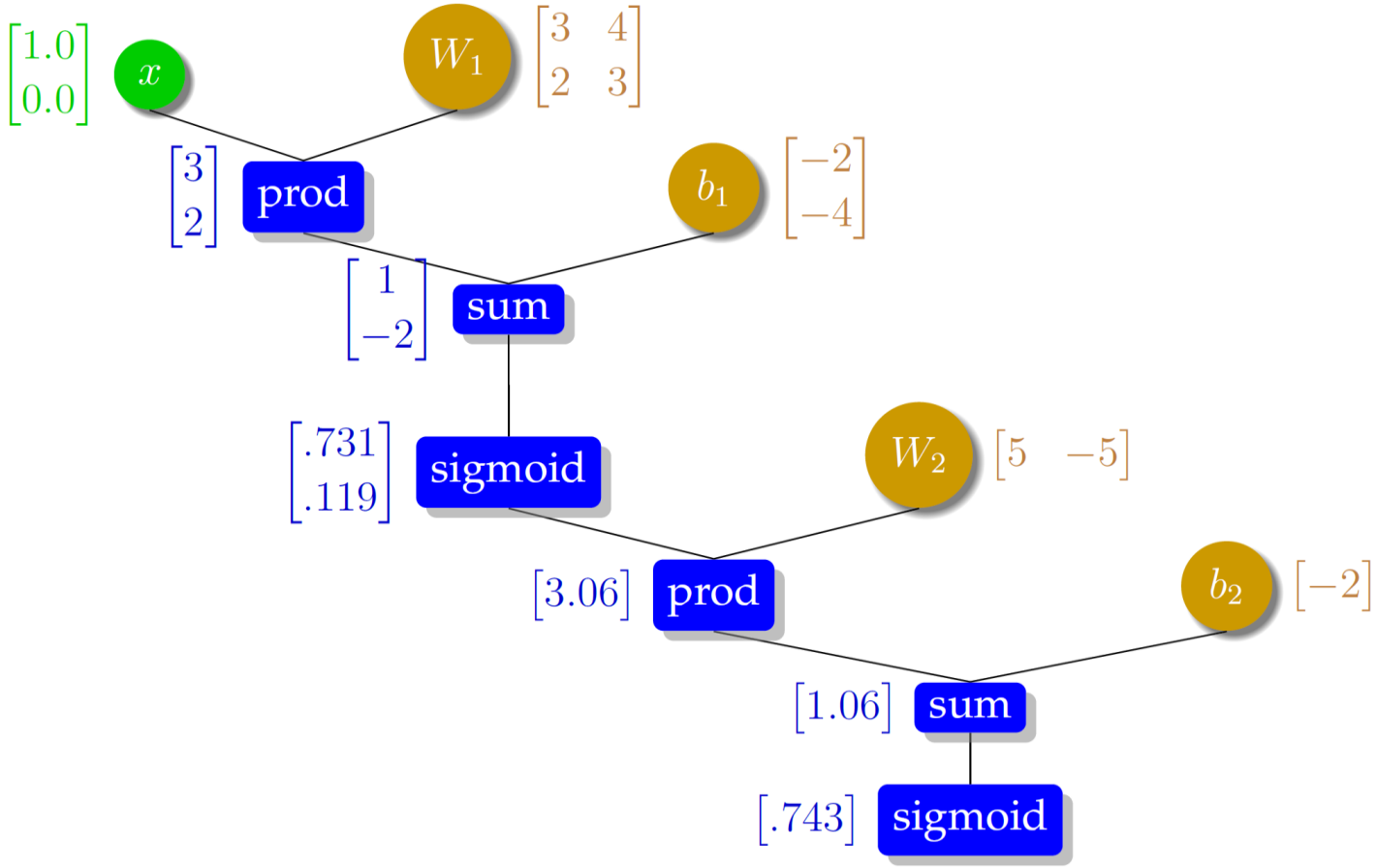


Output table for all possible inputs:

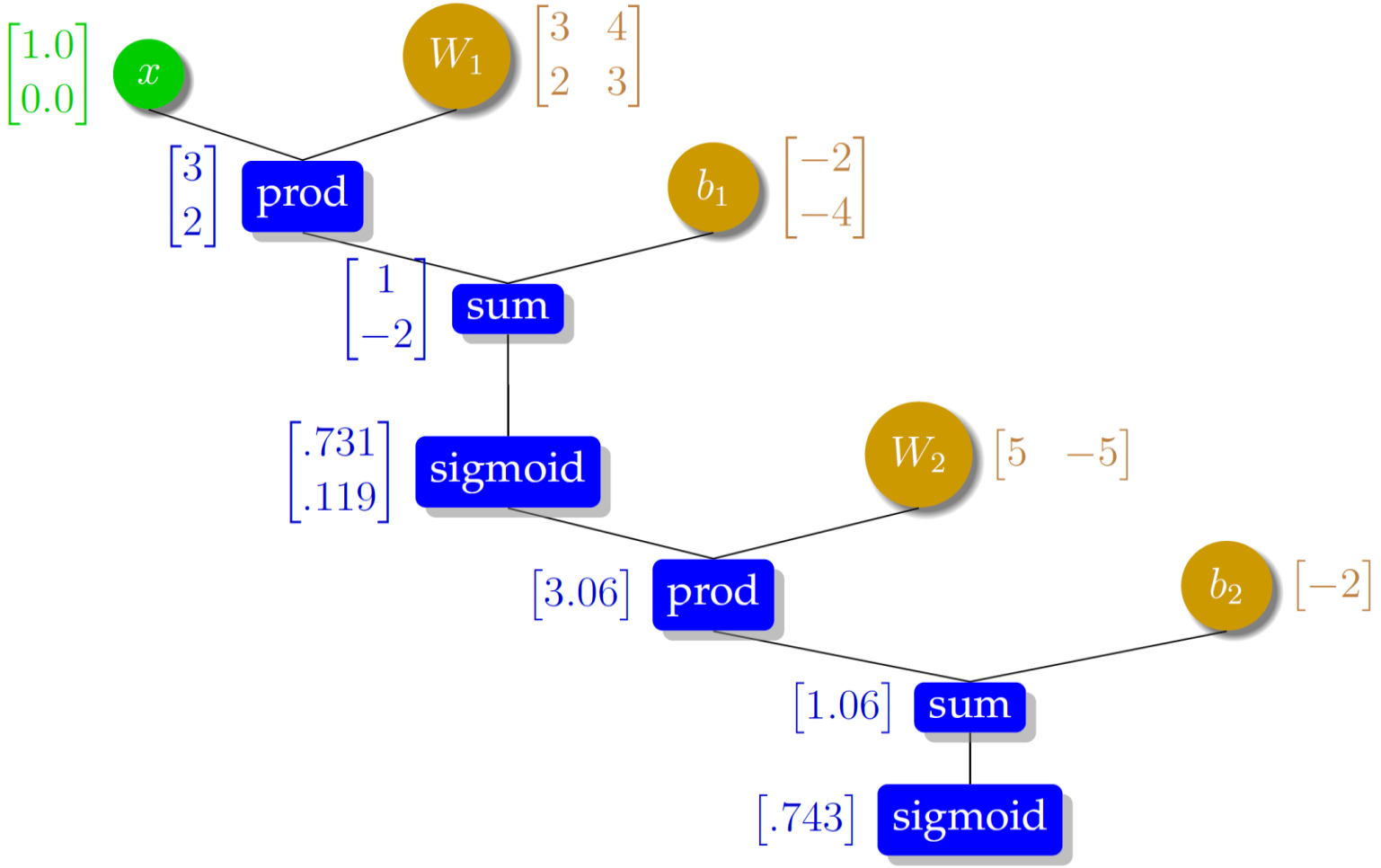
Input x_0	Input x_1	Hidden h_0	Hidden h_1	Output y_0
0	0	0.119	0.018	$0.183 \rightarrow 0$
0	1	0.881	0.269	$0.743 \rightarrow 1$
1	0	0.731	0.119	$0.743 \rightarrow 1$
1	1	0.993	0.731	$0.334 \rightarrow 0$

Neural Networks as Computation Graphs





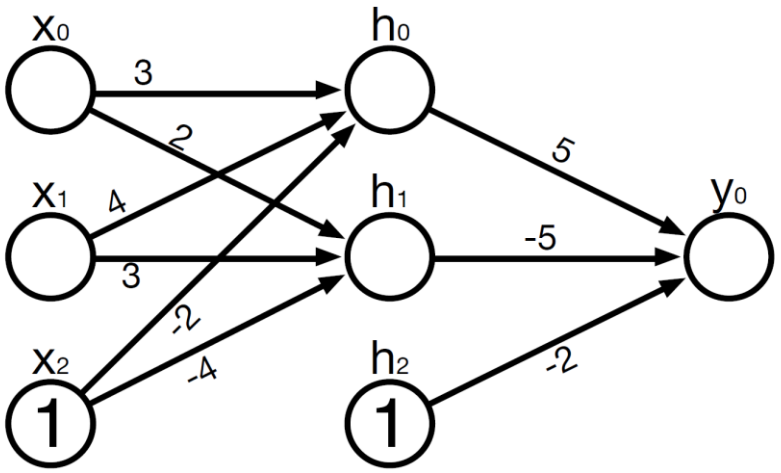
Computation Graphs Make Prediction Easy: Forward Propagation consists in traversing graph in topological order

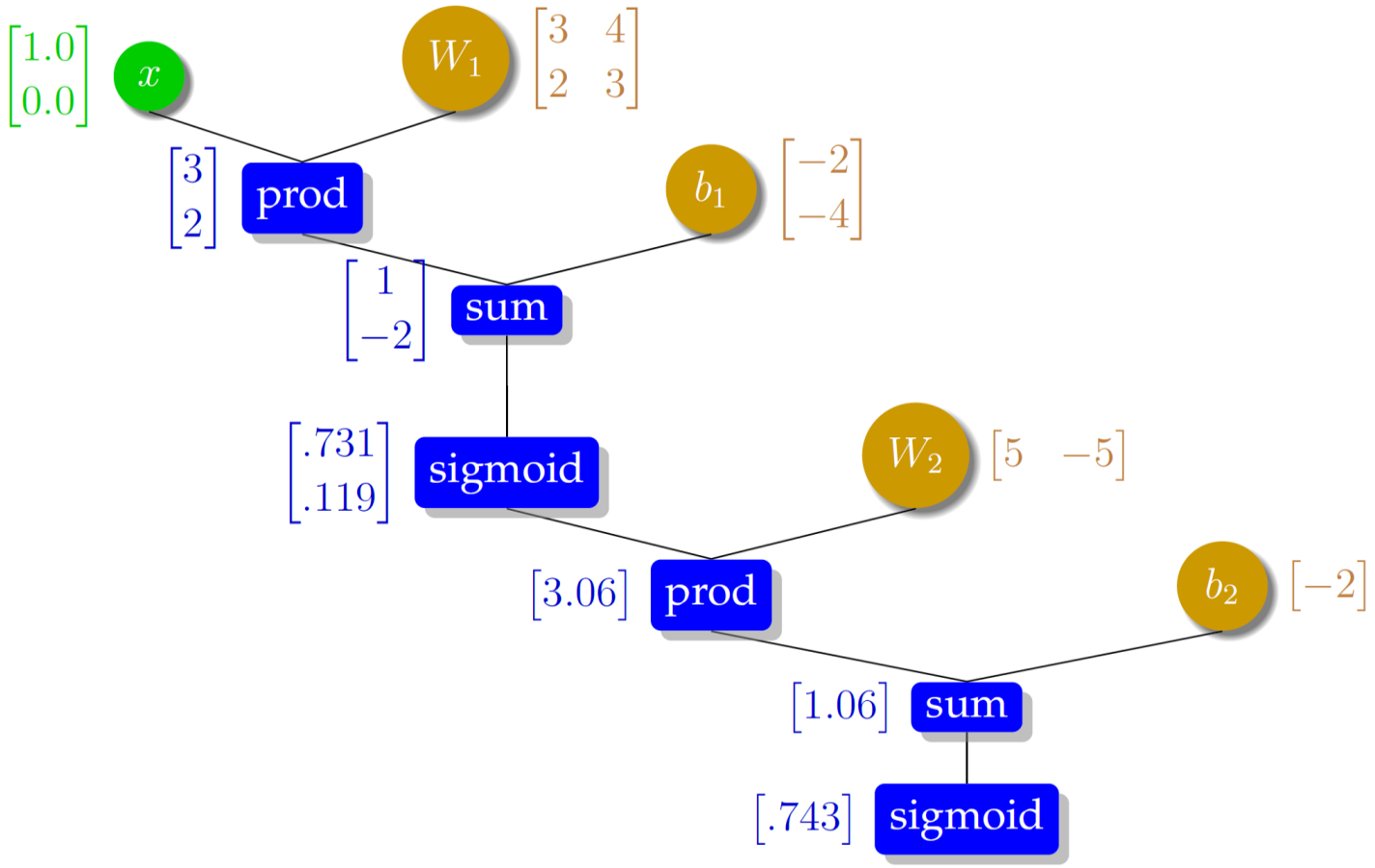


Neural Networks so far

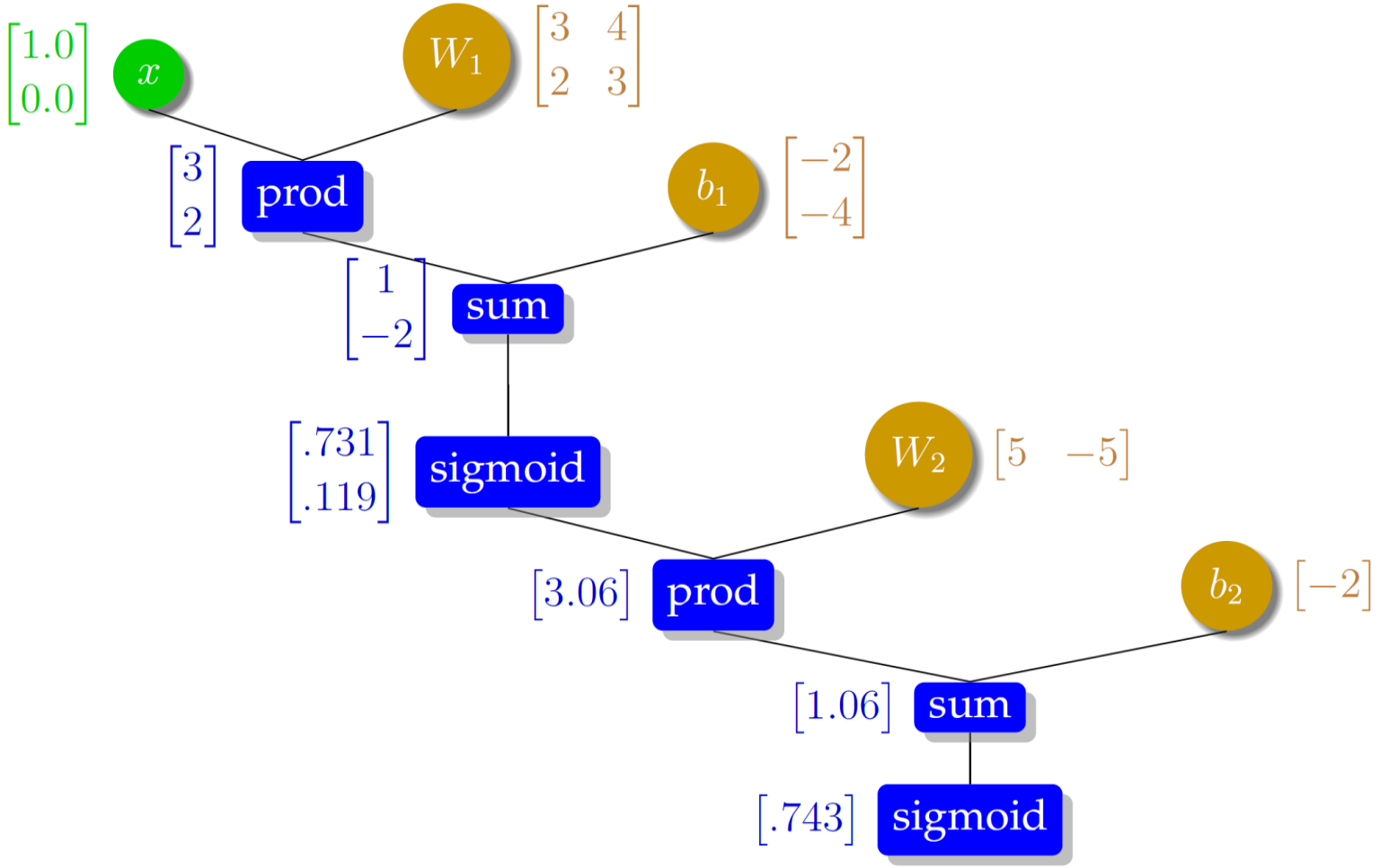
- Powerful non-linear models for classification
- Predictions are made as a sequence of simple operations
  - matrix-vector operations
  - non-linear activation functions
- Choices in network structure
  - Width and depth
  - Choice of activation function
- Feedforward networks
  - no loop
- Next: how to train

Neural Networks as Computation Graphs





Computation Graphs Make Prediction Easy: Forward Propagation consists in traversing graph in topological order



Computation Graph

- Graph contains 3 different types of nodes
  - Parameters of the models (e.g., W1, b1, W2, b2)
  - Input x
  - operations between parameters and input (e.g., product, sum, sigmoid)

- Acyclical directed graph
  - No recursion or loops

- So far each computation node in the graph should consist of
  - A function that executes its computation operation
  - Links to input nodes
  - When processing an example, the computed value

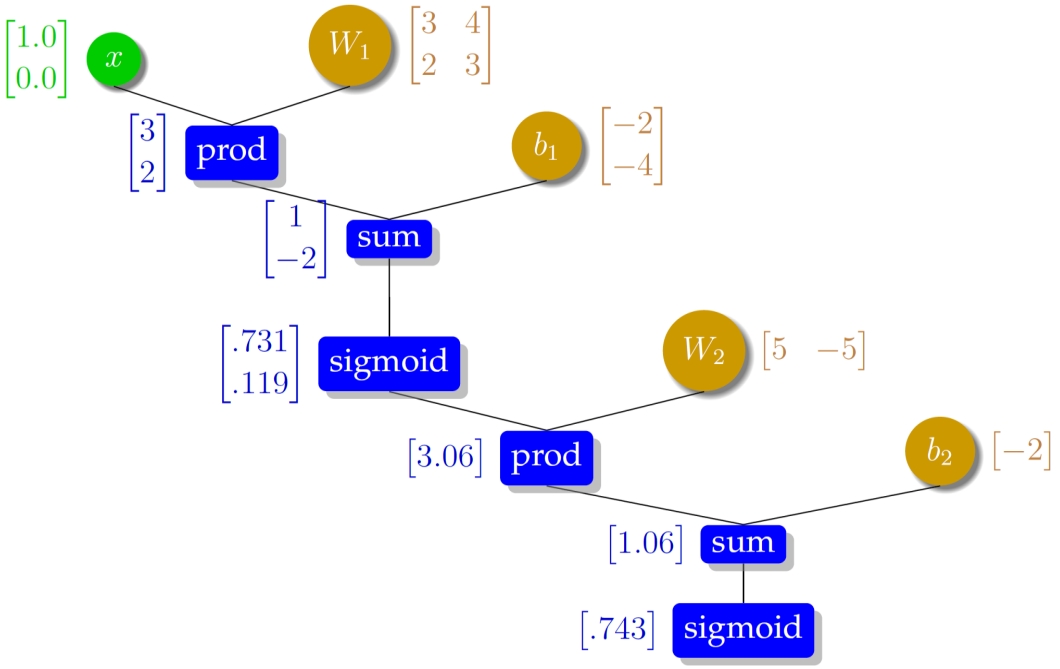
  (we'll add 2 more items to enable training)

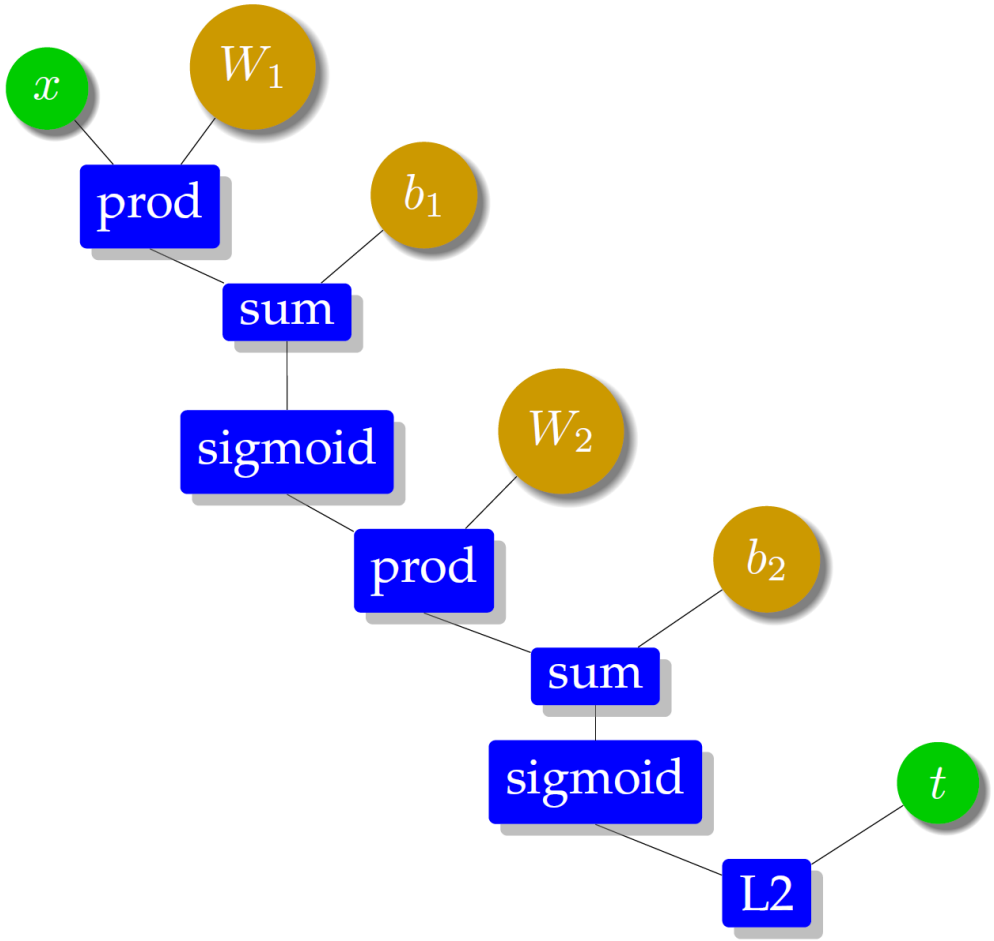
How do we train a neural network?

For training, we need

- Data: (a large number of) examples paired with their correct class (x,y)
- Loss/error function: quantify how bad our prediction y is compared to the truth t
  - E.g. squared error (aka L2 loss) $\text{error} = \frac{1}{2}(t - y)^2$
- An algorithm to minimize the loss: stochastic gradient descent

Extending the Computation Graph to Compute the Loss

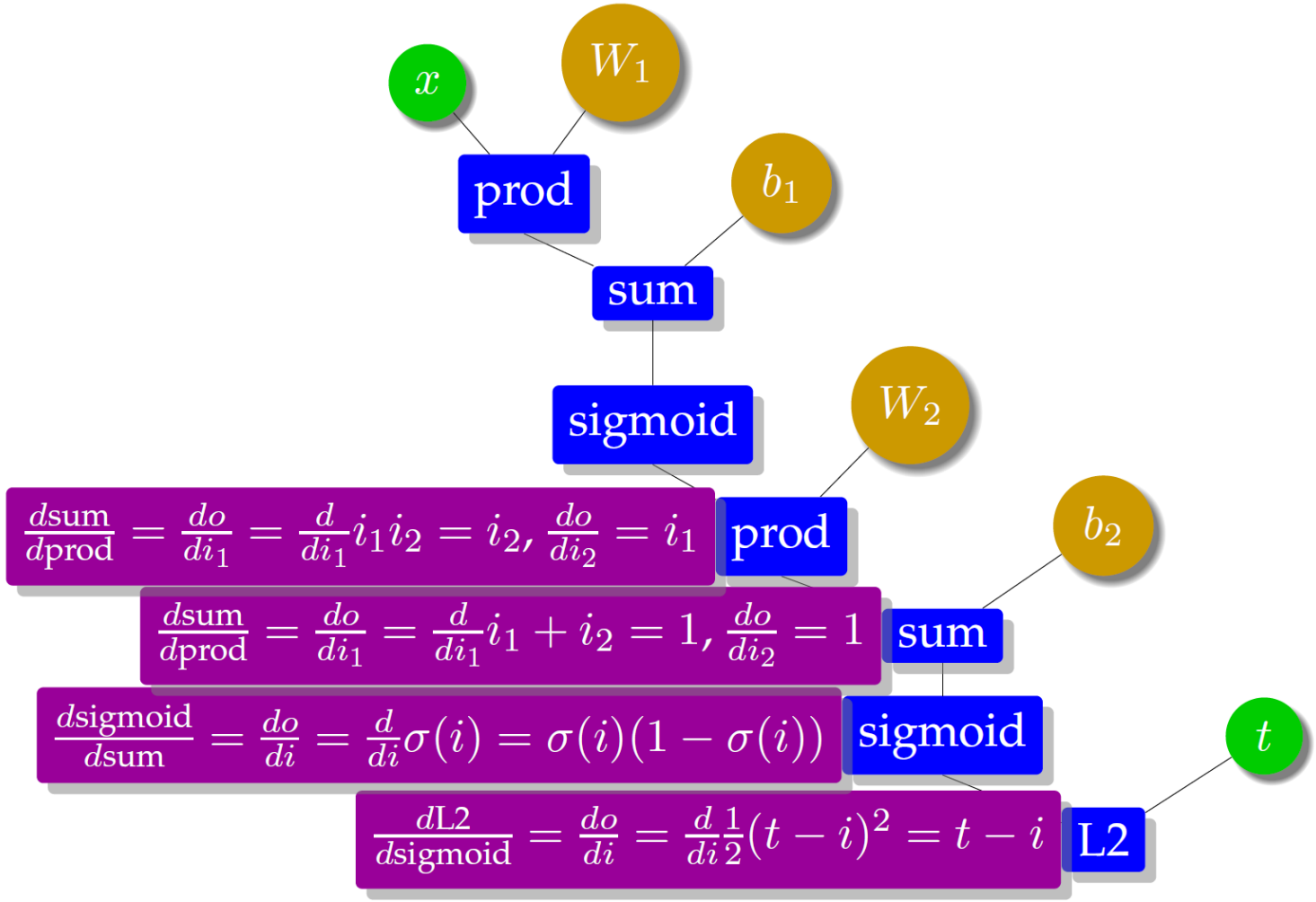




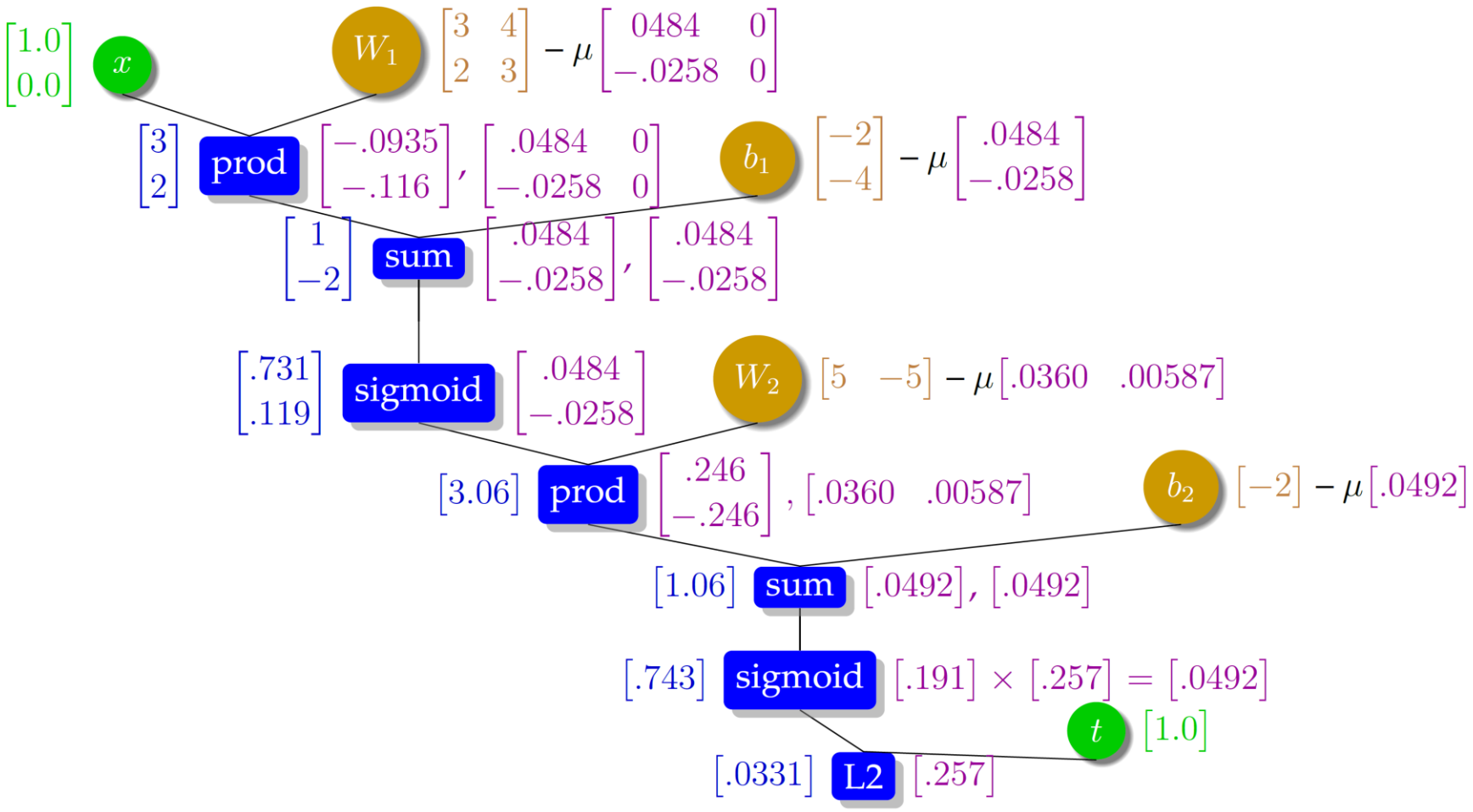
Computing Gradients: Chain rule decomposes computation of gradient along the nodes



$$\frac{dE}{dA} = \frac{dE}{dB}\frac{dB}{dA}$$



Training Illustrated



Computation Graph

- Graph contains 3 different types of nodes
  - Parameters of the models (e.g., W1, b1, W2, b2)
  - Input x
  - operations between parameters and input (e.g., product, sum, sigmoid)

- Acyclical directed graph
  - No recursion or loops

- So far each computation node in the graph should consist of
  - A function that executes its computation operation
  - Links to input nodes
  - When processing an example in the forward pass, the computed value
  - **A function that executes its gradient computation**
  - **Links to children nodes (to obtain downstream gradient values)**
  - **When processing an example in the backward pass, the computed gradient**

Computation Graph: A Powerful Abstraction

- To build a system, we only need to:
  - Define network structure
  - Define loss
  - Provide data
  - (and set a few more hyperparameters to control training)
- Given network structure
  - Prediction is done by forward pass through graph (forward propagation)
  - Training is done by backward pass through graph (back propagation)
  - Based on simple matrix vector operations
- Forms the basis of neural network libraries
  - Tensorflow, Pytorch, mxnet, etc.

Exploiting parallel processing

- Using vector matrix operations helps
  - E.g., if a layer has 200 nodes a matrix operation Wh requires 200 x 200 = 40000 multiplications
  - Can benefit from efficient implementations for Graphics Processing Units (GPU)
- “Minibatch” training by processing multiple examples at a time helps further
  - Compute parameter updates based on a “minibatch” of examples
    - instead of one example at a time
  - More efficient: matrix-matrix operations replace multiple matrix-vector operations
  - Can lead to better model parameters

Neural Networks

- Originally inspired by human neurons, but now simply an abstract computational device
- Can be thought of as combinations of neural units, where each unit multiplies input by a weight vector, adds a bias, and then applies a non-linear activation function
- Or alternatively as a computation graph
- Power comes from ability of early layers to learn representations (i.e. features) that can be used by later layers in the network

Neural Networks

- Choices in network structure
  - Width and depth
  - Choice of activation function
- Feedforward networks (no loop)

- Forward Propagation: predictions are made as a sequence of simple operations
  - matrix-vector operations
  - non-linear activation functions
- Training with the back-propagation algorithm
  - Requires defining a loss/error function
  - Gradient descent + chain rule
  - Easy to implement on top of computation graphs